PROJECT BIBLE · REVISION 6

6DOF Head-Tracking Mods

Resume-ready master reference

Everything needed to build a version-agnostic, full-6DOF OpenTrack head-tracking ASI for any PC game: the core tenets, the universal method, six hooking families, four camera-math conventions, the drive-model taxonomy, the cross-engine camera-data corpus, the standard config, a 22-entry failure catalogue, and a per-game atlas of 51 shipped mods.

Second-pass audit: 51 mods · 345 unique camera tables (882 AOBs) · 49 constant-header games · 820 GPU/scan profiles · 5 CSVs · 51 shipped INIs

July 17, 2026 · **Drop into a new chat and say "continue from this reference"**

Contents

Revision 6 — two new mods shipped, and the traps they exposed. Yakuza 0 and Yakuza Kiwami are in the atlas (§23). The transferable findings matter more than the games: **§17.4** identify a camera by **live data** when the code is encrypted and no static signature exists (an exe 87% INT3 at rest, a 5-byte pattern hitting 93×); **§15.12** the FOV can be a **pair** — raw radians plus a cached **tan(FOV/2)** the renderer actually reads, so writing one does nothing; **§7D** look-at rigs **do** have roll when an explicit up vector is present, and it need not be in a register; and **§25.0.1** grows to **eight** failure modes, adding the three that cost real time here — a signature lifted from a CALL-entered shellcode being **off by 8**, an **inherited sibling constant** that was never re-verified (and corrupts the game), and **a verification that never actually ran** (a dead config key; a stale binary). Also: `.pdata` survives Denuvo intact — 65,628 function records in an exe whose code is 87% encrypted. **§14** no longer names artifacts or quotes raw file counts.

Revision 5 — third-pass audit, aimed at new-mod creation. New: **§15.9** a measured map of **where camera fields live** — 70% within `+0x30`, 94% within `+0x74`, top 8 offsets = 75%, and the camera pointer is in **RCX or RDX 52%** of the time. **§15.10** engine ID from the module name (**22% of games solved by reading the filename**). **§15.11** the FOV-encoding decision table. **§25.0 the first 30 minutes** — the seven questions in cost order, and the five ways a bring-up actually fails. **§26.7** the **de-facto core library that was never extracted** (12 primitives duplicated across 47 mods, `AobScan` with four signatures, the smoothing filter under five names) — and why that, not difficulty, is the measured reason spring smoothing reached only 5/51 mods.

Revision 4 — second-pass audit. New findings: **§15.7** the camera-write **instruction vocabulary** — 61% of every camera hook in the corpus is a single SSE instruction on `[reg+disp]`, and **vtable dispatch is 0.7%**, which retroactively explains the hardest title in the atlas. **§15.8** the constant key *name* declares the camera math (quaternion 44% / look-at 34% / matrix 22%) — matching our own §7 taxonomy. **§15.6** the **steal length is free** (`CONTINUE – START`), removing the most crash-prone step in a code cave. **§21.4–21.6** the config surface is **measurably fragmented** — four spellings of "UDP port", 22 FOV keys — with a canonical **alias table** and a migration rule that renames nothing. **Two corrections to Rev 3:** the table corpus is 596 files but **345 unique**, and the constant-header tree is **duplicated ~4× (49 unique games, not 200)** — the three-layout finding survives with corrected magnitudes (§16). Rev 3's Part III and Rev 1–2 changes retained.

1. Scope & the five core tenets

Flatscreen (mono, TrackIR-style) OpenTrack head-tracking for PC games. An OpenTrack head pose is added **additively on top of the game's own camera** so the view tracks the player's head. Not VR — a headset can be the pose source, but the image stays mono. Delivered as an ASI/DLL loaded next to the game exe.

We add **rotation** (yaw/pitch/roll — look around) and **position** (lean x/y/z — shift the viewpoint). Together: 6DOF.

THE FIVE CORE TENETS

1 — Never take the camera over; nudge it. The game keeps driving its own camera. Never disable the spring-arm, follow-cam, or damping. Reference freecam/VR mods do not disable them either — they override the controller's *output*. Disabling is always the wrong instinct.

2 — Additive and idempotent. Always `SET camera = clean (+) head`, recomputed from the game's clean value each frame. Never `+=`. This makes the apply safe at several sites per frame and impossible to drift.

3 — In-thread, in-frame. Async writes from a worker thread never beat the engine's per-frame rebuild. Apply on the game thread at the instant the engine writes or reads the camera.

4 — Find the camera the renderer actually reads. The obvious camera is frequently not the drawn one. Perfect diagnostics with nothing on screen means you are editing the wrong copy.

5 — Prove the hook before tracking it. Wobble first, on the desktop, with no tracker. If the whole view pans by itself, the hook is right. Everything else is downstream of this.

The three-stage path for every title: (1) Find the camera — signature, chain, or corpus lookup. **(2) Wobble test** — full 6-axis, desktop, no tracker. **(3) Full 6DOF** — rotation, lean, FOV, smoothing, cutscene. Everything is version-agnostic (signature-scanned + INI-driven), so a game patch usually needs only an INI edit.

2. The universal method

The single architecture that works across every engine family in the corpus and every shipped mod. Learn this and the rest is detail.

CAPTURE → GATE → EDIT

```
HOOK 1 — the camera-ADDRESS site        ("which object is the real camera?")
   A dedicated site where the engine resolves/points at the active camera.
   Capture that pointer into a global:            g_cam = <register>

HOOK 2 — the generic TRANSFORM-WRITE site   ("the engine copies a pov into a camera")
   Almost every engine has ONE generic copy/write routine used by MANY cameras
   (render, logic, cull, shadow, reflection, menu, photo).
   -> Hook it once. It fires for all of them.

THE GATE — the whole trick
   if (destination != g_cam) return;      // leave every other camera alone
   apply head pose additively to the transform

WHY IT WORKS
   The generic writer is stable code (good AOB target, version-agnostic).
   The gate makes one hook precise. And because the gate follows whichever
   camera is on screen, it tracks in gameplay AND cutscenes from ONE hook.
```

Independently confirmed three times: the flagship RAGE mod (gate `dest == g_hObj − 0x10` on the generic "copy pov into camera" function with 61 callers); the 41-title engine-sibling table cluster (gate `cmp rcx, [cam]` on the POV copy); and the managed-runtime cluster (gate `cmp rdx, [cam]` on the generic transform writer). Three unrelated engines, one architecture.

The decision sequence

#	Question	Answer determines
1	What engine / anti-tamper?	**Hooking family** (§6). Denuvo/Arxan ban debug registers → sync cave. 32-bit → inline trampoline.
2	Does the corpus already have it?	**Skip discovery entirely** (§14–20). Engine-sibling titles share signatures.
3	How is orientation stored?	**Camera math** (§7). Quaternion / matrix / Euler-degrees / look-at.
4	Is the camera reset or persistent?	**Drive model** (§8). This is where "tracks but feels wrong" is decided.
5	Does the struct write reach the screen?	If no → **GPU constant-buffer injection** (§6E).
6	Do several cameras share the hook?	**The gate** + per-pointer slot table (§8E).

3. Mental model / glossary

Term	Meaning
ASI	A renamed DLL loaded by an ASI loader. Same as a DLL; `.asi` extension.
Proxy / loader	`dinput8.dll` / `version.dll` / `dxgi.dll` / `winmm.dll` / `dbghelp.dll` next to the exe. **Pick a name for an import the game actually has** (§13).
AOB	"Array Of Bytes" — a byte signature of a machine-code site, with `??` wildcards, scanned at runtime so no address is hardcoded.
Code cave / trampoline	Our own machine code the hook jumps into: captures a register, runs the game's "stolen" bytes, calls our C function, jumps back.
Stolen bytes	The instructions overwritten by the `E9` patch, re-emitted in the cave. Must be position-independent or relocated.
VEH	Vectored Exception Handler + hardware breakpoint (DR0/DR7). Fires our handler on the game thread with no code patching.
Idempotent apply	Always SET `camera = clean (+) head`. Safe at several sites per frame; cannot compound.
Sync (in-frame) apply	Applying at the exact instant the game writes/reads the camera, on the game thread → render + culling + shadows all see the same value → no race, no flicker.
The gate	`if (dest != g_cam) return;` — the test that makes one generic hook touch only the real camera. **The central mechanism of the whole project.**
Lean	Positional head translation mapped into the camera's local axes (right/up/forward).
Logic vs render camera	Engines keep a telemetry/logic camera (audio, LOD, script natives) separate from the camera that draws. Same pose, different memory. **The most expensive trap in the project.**
Reset camera	The engine recomputes eye/target from scratch every frame (follow-cam + stick). Needs absolute-on-live (§8).
Persistent camera	The transform is retained state the engine eases each frame. Needs incremental or idempotent-base handling.
Validator / plausibility gate	A range test on struct fields that identifies a real camera (unit-length rows, quat length ≈1, homogeneous w ≈1, roll ≈0). Also the corpus's memory-scan fingerprint (§17).
Wobble test	Self-oscillating camera motion with no tracker, to prove the AOB hooked the right camera before any tracking work.

4. Shared runtime framework

Every mod is the same skeleton.

- **Loader:** ASI-loader proxy next to the exe; mod ships as `{Game}-HeadTracking.asi`. Some titles need a bespoke proxy that forwards a real export (e.g. a `dinput8.dll` forwarding `DirectInput8Create`, or a `dbghelp.dll` forwarding `MiniDumpWriteDump`).
- **Config / log:** reads `HeadTracking.ini`; writes `HeadTracking.log` beside the module, falling back exe-dir → `%TEMP%` → Desktop → `C:\`.
- **Receiver:** Win32 UDP thread, port 4242. Shared `opentrack_receiver.{h,cpp}`, namespace `P5HT`.
- **Hotkeys:** `END` = toggle · `HOME` = recenter. Extras: `PageUp` / `PageDown` / `Insert` = probes/diagnostics.

The wire protocol

```
packet = 6 little-endian doubles, 48 bytes:
  d[0..2] = x, y, z            (centimetres -> metres, x0.01)
  d[3..5] = yaw, pitch, roll   (degrees)

if (n >= 48) { memcpy(d, buf, 48); ... }              // accept >=48; ignore trailing fields
if (isfinite(all 6)) { store; m_lastRecvMs = now; }   // NaN/inf from a mis-set tracker rejected
IsReceiving() == running && (now - lastRecv) < 500ms
```

Receiver craft worth preserving

- `SO_REUSEADDR` **deliberately NOT set.** With it, a second receiver in the same process could also bind 4242 and steal packets. Without it a stray second bind fails and the first receiver keeps the data. (UDP has no TIME_WAIT, so quick rebinds still work.)
- `SIO_UDP_CONNRESET` **disabled** via `WSAIoctl(s, 0x9800000C, &FALSE, ...)` — stops a spurious ICMP port-unreachable killing the recv loop.
- **Bind** `INADDR_ANY`, not loopback — supports remote/phone trackers.
- `SO_RCVTIMEO` **= 200 ms** so the thread observes the stop flag and exits cleanly.
- **Start the receiver only after the hook installs** — so only the real game process binds 4242.
- **Single-instance mutex** in `DllMain` — a double-load would put a second receiver on 4242.

Recurring supporting craft (copy into every new mod)

- **Recenter from the first REAL packet**, never the all-zero default — otherwise the resting pose bakes in an offset and the view starts rotated.
- **Dummy-camera rejection** — skip near-origin position, non-unit quaternion, or non-orthonormal matrix objects sharing the write site.
- **Last-good cache** — validate the captured object; if a transient capture is bad, fall back to the last valid one so tracking never stalls.
- **Warm-up gate** — require a camera to look valid and stable for N frames (8 is the standard) before the first write.
- **No modal MessageBox, ever** — it wedges a fullscreen DX game.

Shared file layout of a mod package

```
{Game}-HeadTracking/
  {Game}-HeadTracking.asi       # the mod
  dinput8.dll (or version/dxgi/dbghelp)  # loader proxy
  HeadTracking.ini              # all tuning + the version-agnostic [Hook] block
  license.txt
  README.md
  src/
    dllmain.cpp                 # the whole mod (per-game camera logic)
    opentrack_receiver.{h,cpp}  # shared
    logger.h                    # shared
    build.sh / build.bat
    (inlinehook_x86.h)          # reusable x86 inline-hook primitive
    (d3d11_hook.cpp)            # GPU CB injection
    (proxy_*.cpp + *.def)       # when shipping a bespoke proxy
```

Framework drift (measured). `opentrack_receiver.h` exists in **8 distinct variants** and `logger.h` in **8**. The richest receiver adds `LatestPredicted(nowMs, capMs)` (velocity extrapolation) and `Packets()` (rate diagnostics). These files are nominally "shared, byte-identical" but are not. Consolidating on the richest variant is a standing cleanup.

5. Build pipeline & packaging

Toolchain: MinGW-w64, win32 threads variant. The posix variant links a larger runtime and produces a different-sized binary — confirmed by rebuilding a shipped source: win32 reproduces the exact 384,690-byte `.asi`; posix gives 536,252. Use the `-win32` alternatives to reproduce exact sizes.

```
# 32-bit games (x86)
i686-w64-mingw32-g++-win32 -O2 -std=c++17 -shared -static -static-libgcc -static-libstdc++ \
  -o {Game}-HeadTracking.asi dllmain.cpp opentrack_receiver.cpp -lws2_32 -lwinmm -luser32

# 64-bit games (x64)
x86_64-w64-mingw32-g++-win32 -O2 -std=c++17 -shared -static -static-libgcc -static-libstdc++ \
  -o {Game}-HeadTracking.asi src/*.cpp -lws2_32 -lpsapi

# with a D3D11 CB-injection hook   add: -ld3d11
# with a DX12 present hook         add: -ld3d12 -ldxgi
```

`-static*` bakes in the runtime so no MinGW DLLs are needed on target. Add `-lwinmm` for `timeBeginPeriod`, `-luser32` / `-lpsapi` for `GetModuleInformation`. Debian/Ubuntu packages: `g++-mingw-w64-i686`, `g++-mingw-w64-x86-64`.

MinGW has no SEH. `__try/__except` is unavailable. For crash-safe access to game memory use `ReadProcessMemory` / `WriteProcessMemory`, or a vectored-exception guard with a scoped flag + `longjmp` (§13).

Packaging. A complete deliverable is a zip containing the `.asi`, the proxy DLL, `HeadTracking.ini`, `license.txt`, `README.md`, and the full `src/`. Verify the rebuilt binary size before shipping.

6. The six hooking families

Which family to use depends on the engine's anti-tamper and where the camera is touched.

§	Family	Use when	Representative mods
5A	x64 synchronous code-cave	Default for x64. Denuvo present (debug registers banned).	ACO, SOTTR, ROTTR, Crimson, Guardians, Alan Wake 1/2, Dead Space, NieR Replicant, Ni no Kuni, RDR2
5B	HW breakpoint + VEH	Anti-tamper that dislikes code patches but allows DRs. No bytes patched.	Yakuza LAD, AC Unity (fallback), P5R (fallback), Batman AA, P4G
5C	x86 inline trampoline	32-bit titles.	Dragon Age 2, Batman AC/Origins, Witcher 2, RE6, MGR
5D	UE3 writer-hook + auto-offset	Unreal Engine 3 POV memcpy.	Batman Arkham Origins / City
5E	D3D11 constant-buffer injection NEW	The struct write never reaches the screen; engine rebuilds matrices every frame.	Deus Ex: Human Revolution V2
5F	Native scripted camera NEW	The game exposes a camera scripting API (GTA).	GTA V Enhanced (Mode 5)

6A. x64 synchronous code-cave (Denuvo-safe) — the workhorse

Denuvo stalls on debug registers, so instead of a HW breakpoint we steal the bytes at the camera write/read site into a cave. Canonical construction (from SOTTR):

1. **AOB-scan executable regions only** — walk `VirtualQuery`, test `PAGE_EXECUTE*`.
2. `AllocNear(target)` — `VirtualAlloc` a page within ±2 GB of the site so an `E9 rel32` reaches it.
3. **Cave body** (hand-emitted bytes) — see skeleton below.
4. **Patch the site** — `VirtualProtect` RWX, write `E9 rel32` + NOP pad (steal−5 NOPs), restore protection, `FlushInstructionCache`.
5. **Second, capture-only cave** at the camera-address-select site (IGCS `CAMERA_ADDRESS_INTERCEPT`) to grab the ONE gameplay camera pointer, so the apply touches only it — rejects menu/dummy cameras sharing the write site (which cause start-tilt / shake / dead lean).

```
; --- capture base register (e.g. rcx) ---
push rcx
push rax
mov  rax, &g_camBase
mov  [rax], rcx
pop  rax
pop  rcx
; --- stolen bytes (game writes/reads its camera) ---
<original N bytes>
; --- save state, align, shadow space, call C apply ---
push rax/rcx/rdx ; push r8..r11 ; push rbp ; mov rbp,rsp
and  rsp,-16          ; 16-byte align (do NOT assume entry alignment)
sub  rsp,0x60
movups [rsp+0x00..0x50], xmm0..xmm5
sub  rsp,0x20         ; Win64 shadow space
mov  rcx, &g_camBase
mov  rcx,[rcx]        ; arg0 = camera base
mov  rax, &CaveApply
call rax
add  rsp,0x20
movups xmm0..xmm5, [rsp+..]
mov  rsp,rbp ; pop rbp ; pop r11..r8 ; pop rdx/rcx/rax
; --- jmp back (absolute; immune to rel32 range) ---
FF 25 00000000 <qword site+steal>
```

Ordering: stolen-bytes-first vs apply-first

This choice is **not** cosmetic — it decides whether the edit affects the current frame.

- **Stolen-first** (SOTTR, Guardians, Alan Wake, Ni no Kuni 4th pass) — the game writes its clean camera, then we post-modify it. Use when hooking a **write** site. Gives a guaranteed-clean base.
- **Apply-first** (Ni no Kuni 1st design, RDR2) — we edit the source data *before* the game consumes it. Use when hooking a **read** site, or when the stolen bytes themselves read the values we want to change. Ni no Kuni's cave runs Apply first precisely because the stolen `movaps xmm5,[r9]` (target) and the resume `subps [rcx+0xE0]` (eye) READ the points Apply edits.

> **rsp-relative stolen bytes.** If a stolen instruction is rsp-relative (e.g. `lea r8,[rsp+0x30]`), your register save/restore must be **net-zero on rsp** at the point the stolen bytes run, or the displacement is wrong. Ni no Kuni documents exactly this.

Rate-limiting (Dead Space refinement)

The write site can fire many times per frame. A worker thread sets a one-byte `g_applyGate` at ~333 Hz; the cave does `cmp byte [gate],0 / jz skip / mov byte [gate],0` so the expensive apply runs at a bounded rate while frequent hits stay a cheap skip.

`AllocNear` must walk free regions FIX

The naive version blind-`VirtualAlloc`s at candidate addresses across ±1 GB, which fails on reserved pages. Around a densely-mapped game it finds nothing near, falls back to far memory, and the `E9` is out of range ("cave out of rel32 range (~-5GB)" — the Alan Wake Remastered failure). Correct version:

- Walk `MEM_FREE` regions via `VirtualQuery` across the full ±~2 GB (down, then up), allocating in the first free slot in range.
- **Fallback:** if nothing is within 2 GB, patch with a **14-byte** `FF 25` **absolute jump**, growing the steal across trailing NOP padding to a clean instruction boundary. Install can then never fail for range reasons.

6B. Hardware breakpoint + VEH (no code patch)

No bytes are patched — a debug-register breakpoint fires our handler on the game thread.

- `AddVectoredExceptionHandler(1, veh)`; the handler is inert until a BP is armed.
- **Arm across all threads:** snapshot threads (`TH32CS_SNAPTHREAD`), `SuspendThread`, `GetThreadContext(CONTEXT_DEBUG_REGISTERS)`, set `Dr0 = addr`, `Dr7 |= 1` (local enable); for a write BP also `Dr7 |= (1<<16) | (3<<18)` (RW=write, LEN=4). Execute BPs leave those clear. `SetThreadContext`, `ResumeThread`.
- **Handler:** on `EXCEPTION_SINGLE_STEP`, read the camera from the captured register (`ContextRecord->Rcx`), apply, clear `Dr6`, and for an execute BP set the Resume Flag (`EFlags |= 0x10000`) so the instruction runs once without re-triggering. Return `EXCEPTION_CONTINUE_EXECUTION`.
- `DR6 & 0x1` distinguishes our DR0 watch firing from someone else's.

Three hard gotchas. (1) **Arming during loading crashes the game** — use a no-breakpoint gameplay detector (wait for a moving/real camera) before arming, and re-arm periodically for new threads / map changes. (2) **Denuvo and Arxan ban debug registers** — on GTA V Enhanced, touching DR0–DR7 tears the process down. Use 5A. (3) **Never combine a VEH data-breakpoint with a working function hook** — see the AC Unity choppiness fix below.

AC Unity choppiness — a cautionary fix. The mod installed *both* a clean per-frame vtable function-hook *and* DR0/DR1 data breakpoints. AC Unity writes the camera many times per frame, so the data breakpoints fired a CPU-exception storm into the handler every frame (low fps) **and** applied the head pose a second time on top of the function hook (judder / compounding). **Fix:** the HW-BP/VEH path is now a **fallback only**, arming solely when the vtable hook failed to install. One apply, once per frame, in-band, no exceptions.

6C. x86 inline trampoline

Overwrite a site with `E9 rel32` → detour; the detour preserves state, calls the handler, runs stolen bytes, jumps back.

- Preserve GP regs + flags with `pushad` / `pushfd`.
- **Preserve x87 state with `fnsave` / `frstor`** — default MinGW x86 uses x87 for float; omitting this corrupts the game's FP state. This is the classic x86 mistake.
- `inlinehook_x86.h` (Witcher 2) is the clean reusable primitive: a compact instruction-length decoder (`ilen`) so it steals whole instructions, a thread-suspender (suspend other threads while patching), and a trampoline that fixes a leading rel32 call/jmp if one is stolen.

6D. UE3 writer-hook with runtime auto-offset

The engine `memcpy`s the camera POV (`FVector Location` + `FRotator Rotation`) every frame. We inject after the copy and add the pose.

- A separate **FOV-read hook captures the live camera pointer** (`ecx`) — the FOV write only touches the gameplay camera, so it is the reliable way to identify it (also used by Witcher 3).
- **Auto-discover the POV offset:** capture the writer's destination; if `dest − cam < 0x2000` it's inside the camera object → derive the offset at runtime instead of hardcoding it. The most version-proof mechanism in the library.
- Caves are hand-assembled via an `Emit` byte-emitter with conditional-jump fixup slots (`0F 8x rel32` written with a placeholder, patched once the target label is known).
- **UE units:** `FRotator` are ints (65536 = 360°), `FVector` are floats; lean is rotated by camera yaw.

6E. D3D11 constant-buffer injection NEW

The escape hatch when memory writes don't reach the screen. Implemented in Deus Ex: Human Revolution V2 and the answer to the P4G blocker.

The engine rebuilds the camera matrices in memory every frame, so writing them from a polling thread races the rebuild (flicker). Instead, inject at the GPU boundary: hook the constant-buffer upload, and the instant before the GPU consumes the buffer, find the camera VIEW matrix inside it and rotate it by the head pose. **The game's own memory is never touched** — no race, no flicker, no feedback.

```
Throwaway D3D11 device -> read the process-wide ID3D11DeviceContext vtable -> patch slots:
   Map              idx 14
   Unmap            idx 15
   UpdateSubresource idx 48    // many engines push CBs this way instead

hkMap:   hr = oMap(...); record {resource -> {ptr,size}} in a map;  return hr
hkUnmap: scan the recorded buffer for the view matrix; rotate in place; oUnmap(...)
```

Finding the matrix inside an opaque buffer

The buffer is untyped bytes. Match candidate 4x4s against the **live view matrix read from the game's pointer chain** — using **rotation AND translation**, so it cannot be confused with the world matrix:

```
matchView(m, rot, tr) -> 0 = no match
                         1 = direct    (row-major as in memory)
                         2 = transposed (column-major upload)
  rotation tolerance e = 0.03
  translation tolerance te = |tr[0]|*0.03 + 8.0   // scale-relative

apply, direct:    M := M * Rh4   (post) -> each row's xyz *= Rh, 4th component kept
apply, transposed:M := Rh4^T * M (pre)  -> rows 0..2 = Rh^T * rows; row 3 kept
```

DXHR's chain for the match: `CameraManager = *(moduleBase + 0x187CEC0)` → `CurrentCamera = *(CameraManager + 0x30)` → `view matrix = CurrentCamera + 0x80`. Buffer size gate: `MINBYTES=48, MAXBYTES=65536`.

When to reach for 5E. When the §13 signature "perfect diagnostics, nothing on screen" persists after you've confirmed you're on the render camera and applying in-thread. It is also the only route to **roll on a look-at rig** (§7D) and the documented next step for P4G and a matrix-level NieR Automata build.

6F. Native scripted camera NEW

When the game exposes a camera scripting API, the safest mod is no memory writes at all. GTA V Mode 5: create a scripted camera with the natives (`CREATE_CAM_WITH_PARAMS`, `SET_CAM_COORD`, `SET_CAM_ROT`, `RENDER_SCRIPT_CAMS`) and drive it each frame from a mirror of the gameplay camera plus the head offset. Natives are only safe on ScriptHookV's script fiber, so all of it runs there.

- **Result:** full stable 6DOF in all normal gameplay — on foot, in vehicles, aiming, first/third person. No memory writes, no anti-tamper risk.
- **Wall:** during story cutscenes the cutscene *director* owns the camera; a scripted camera cannot override it. This is a hard limit of the native API and is what forced the memory work.
- **Value:** keep it as a fallback mode. If the AOB ever misses, the mod degrades to Mode 5 rather than to nothing.

7. Camera math — four conventions

The apply math depends on how the game stores orientation. Detect which from the struct.

7A. Quaternion camera

Fields: position vec3, quaternion (x,y,z,w), fov.

```
// compose head rotation onto the game quaternion (order picks local vs world)
Q qn = order ? qnorm(qmul(qhead, gameQ))    // world: q_head * q_cam
             : qnorm(qmul(gameQ, qhead));   // local: q_cam * q_head

// lean in camera space, rotated by the game quaternion, added to position
float wl[3]; qrotvec(gameQ, leanLocal, wl);
P[0]+=wl[0]; P[1]+=wl[1]; P[2]+=wl[2];
```

Build the head quaternion **per-axis** so each head angle turns about an assigned game-quat axis (0=X, 1=Y, 2=Z); `qfromEuler(yaw,pitch,roll,mode)` handles Y-up (mode 0) vs Z-up (mode 1) engines. Dummy-camera gate: quaternion length in [0.5, 1.5], position magnitude above a floor.

7B. VIEW / WORLD matrix camera (4×4)

Rotate about the camera's own basis, recompute the translation:

```
// clean base captured at the write; R = head rotation (about camera-local axes)
mul4(baseW, R, newW);   // WORLD' = WORLD_base * R   (view->world)
mul4(Rt, baseV, newV);  // VIEW'  = R^T * VIEW_base  (= inverse(WORLD'))

// eye = WORLD 4th column; lean along the ROTATED basis (right/up/fwd = newW cols)
ex += lx*rx + ly*ux + lz*fx;  /* ...ey, ez... */
newW[3]=ex; newW[7]=ey; newW[11]=ez;

// keep VIEW consistent: translation = -(R_view . eye)
newV[3] = -(newV[0]*ex + newV[1]*ey + newV[2]*ez);  /* ...7, 11... */
```

- If the game stores **only a single VIEW matrix** (world→camera), **negate both** the rotation angles and the lean translation — the view matrix moves the world oppositely to the eye.
- Some engines keep a matrix **plus a duplicate** rot+pos copy (Dragon Age Inquisition: ROT@0x00 / POS@0x30 and ROT2@0x40 / POS2@0x70) — **write both** or the renderer reads the stale one.
- `transpose` INI toggles rows-vs-columns basis.
- **Stride is not always 0x10.** Northlight packs a tight 3×3 (stride `0x0C`: row0@+0x00, row1@+0x0C, row2@+0x18, pos@+0x24). Assuming 0x10 makes "row2" start at +0x20 and collide with the position — the orthonormality check then fails and nothing is written. Expose `RotStride`.
- **Not always floats.** Quantum Break's camera→world matrix is **12 doubles** at `camStruct+0x70`.

Split-axis rotation (Ni no Kuni 6th pass) NEW

A subtle, high-value finding. Applying all three head axes on one multiply side makes yaw *or* roll skew diagonally when the camera is pitched. Verified numerically on real orientations, position preserved exactly:

```
yaw   -> LEFT  (Ry * M) : pans about world-up; right vector stays horizontal
pitch -> RIGHT (M * Rx) : tilts up/down about screen-right; horizon stays level
roll  -> RIGHT (M * Rz) : pure horizon tilt; look direction unchanged

final:  A' = Ry * A * (Rx * Rz),  translation recomputed position-preserving
        (recover pos from A,t; re-encode with A')
```

Idempotent base (matrix example, change-detected)

```
if(!g_have){ memcpy(g_clean, live, 64); g_have = true; }
else { float d=0; for(int i=0;i<16;i++) d += fabsf(live[i] - g_lastWritten[i]);
       if(d > 1e-4f) memcpy(g_clean, live, 64); }   // engine wrote a fresh clean frame -> refresh base
// ... compute newM from g_clean + head ...
memcpy(M, newM, 64); memcpy(g_lastWritten, newM, 64);
```

7C. Euler-degrees camera

Witcher 3 stores orientation as **degrees**, not a quaternion:

```
cam+0x80 = Roll  (degrees)
cam+0x84 = Pitch (degrees)
cam+0x88 = Yaw   (degrees)
cam+0x70 = position (left untouched)
```

Proven at runtime: the forward vector equals `(-sin(yaw), cos(yaw), ...)` of `cam+0x88`, and the "quaternion-looking" field at +0x70/74/78 logged as `323.6 / 280.5 / 17.5 / 1.0` — degrees, with +0x7C a constant 1.0. Just add the head angles to the Euler triplet. **The classic wrong turn is treating +0x70 as a quaternion.**

7D. Look-at (eye/target) rig NEW

Whether roll exists depends on ONE question: is there an explicit up vector? NEW
The blanket rule "look-at rigs have no roll" is wrong — it holds only for rigs that pass *eye* and *target* and derive up from a hardcoded world axis. **If the engine hands you an up vector, roll is fully available**: rotate the up about the view direction (Rodrigues) and the rig tilts. Two titles decoded on one engine both expose an up vector in the camera-build path, and both do full 6DOF including roll.
How to tell in one frame: dump the candidate vectors and look for a **unit-length** vec3 alongside two world points. A unit vector that is not a constant world axis, and that *changes* as the camera banks, is an explicit up. If the only unit vector is a fixed world axis, the rig has no roll — ship `RollSensitivity=0` rather than fake it.
The up vector need not be in a register. One title holds focus and eye in xmm registers but keeps **up in memory**, addressed by a pointer register — the cave reads it, edits it, writes it back. Do not assume the whole rig arrives the same way.

A whole convention Rev-1 never described. The camera is stored not as an orientation at all, but as two world points; the engine derives the view from them.

Game	Eye / target	Notes
NieR: Automata	eye @ +0xF0, target @ +0x100	distance @ +0x134 == |eye−target|; Y-up
Ni no Kuni Remastered	eye @ +0xE0, target @ +0xF0	tilt/up param @ +0x110
Yakuza Kiwami 2	pos @ +0x10, focus @ +0x20, up @ +0x30	fov @ +0x58 (radians); explicit up vector
Metal Gear Rising	eye @ +0x1B0, lookat @ +0x1C0	Y-up; camera = EAX at `fld [eax+0x90]`

The drive model

```
Rotation: ORBIT the eye about the target by head yaw/pitch  (length-preserving)
          -> the character stays framed while you look around
Lean:     translate eye (and optionally target) by head x/y/z
```

A look-at rig has NO ROLL degree of freedom. "Up" is derived from world-up, so head roll cannot be expressed by editing eye/target — full stop. Do not waste time on it. Confirmed on NieR Automata by tracing the whole camera-finalize function (`0x73E000..0x73E3F2`): it interpolates eye/target from multiple endpoints, derives `view = eye − target`, writes distance to +0x134, lerps FOV — and **never reads the euler fields at +0x40/44/48 and never constructs an up-vector**. Roll exists only in the final view-projection matrix uploaded to the GPU. **The only path to roll on a look-at rig is family 5E (D3D11 CB injection).**

Corollary: if the struct *does* carry euler fields, check whether they're consumed **upstream** of your hook — on NieR they are, so writing them is a silent no-op.

The distance-system trap NEW

A look-at engine usually owns a camera-distance/collision system that reacts to the eye moving. NieR's live log was decisive: with small lean the two distance fields agreed (`+0x50 == +0x134 ≈ 4.5`), but on a hard lean (`head x=0.29, z=0.28`) with `WorldUnitsPerMetre=10` the eye moved ~2.8 units, and `+0x50` spiked to **2500** while `+0x134` stayed 4.47. That distance system then fights our per-frame eye write → in/out oscillation = shake. **Fix:** reduce `WorldUnitsPerMetre` (10 → 1.0) so lean never displaces the eye far enough to trip the clamp, and add a `LeanDeadzone` (0.02 m) to kill resting-bias creep and micro-shake.

8. The drive-model taxonomy

This section is the most valuable single discovery in the library and was absent from Rev-1. **Getting the hook right is only half the job — the drive model must match how the engine treats the camera as state.** The wrong model produces "tracks but feels wrong" symptoms that look like tuning problems and are not.

8A. The fundamental question: persistent or reset?

Type	Behaviour	Correct model
Reset	Engine recomputes eye/target from scratch every frame (follow-cam behind the character + right stick). Distance locked. Tracks the character across the world.	**Absolute-on-live (7B)**
Persistent	The transform is retained state the engine eases each frame toward a goal.	**Incremental delta**, or idempotent clean base (7C)
Freshly-written	The engine writes a clean transform at a site we hook, then reads it back this frame.	**Idempotent clean base (7C)**

How to tell them apart from a log

- **Reset:** eye/target values differ wildly frame-to-frame as the character moves, but |eye−target| is pinned at a constant (~4.0 on Ni no Kuni, ~15.0 on NieR).
- **Persistent:** writing an absolute value *pins* the camera until the game's drift crosses your change-detection threshold, then it **snaps back** — the signature symptom.

8B. Absolute-on-live — the model for reset cameras

The rule. Each frame, re-apply the **FULL head angle** to the game's **FRESH** eye/target.

- Head centred ⇒ identity ⇒ we write back **exactly** what the game produced. Follow-cam and right stick are untouched (verified as a numerical no-op on real frames).
- Head turned ⇒ the same angular offset is re-applied to whatever fresh value the game gives, so the look **persists while held** and the camera still follows underneath it.
- **No stored base, no accumulation, no change-detection** — nothing to rebase, nothing to drift.
- Because there is no accumulation, the offset is bounded purely by `LimitDeg`, so sensitivity can be a real value instead of being throttled to hide drift.

The three failed models that led here (Ni no Kuni's log is the definitive account — five passes):

1. **Absolute + change-detection rebasing.** Fought the follow-cam: pinned the camera at our last write until the game's drift crossed the threshold → **snap-back**, and starved the right stick.
2. **Incremental (delta-only).** Correct for a persistent camera, wrong here: the game's per-frame reset **discards** the accumulated rotation, so head-look never persisted ("fixed camera"), and the follow-cam absorbed the incremental lean ("tiny, barely noticeable movement").
3. **Shared change-detection base across camera objects.** Two cameras (gameplay + a fixed reflection/cubemap rig at distance exactly 50) flowed through one hook and thrashed one global base: both rotation and lean accumulated, the gameplay camera's eye→target distance collapsed 4.00 → 0.70 → 0.39, producing **spin** (look vector flips as eye meets target) and **tilt**. Fixed by 7E.

NieR Automata reached the identical conclusion independently: its prior build had to throttle look to **0.1** and felt weak — root cause was the drive model, not the hook. Replacing change-detection rebasing with absolute-on-live let sensitivity return to a real value.

8C. Idempotent clean base — the model for freshly-written cameras

The default for family 5A stolen-first hooks. Capture the game's clean camera the instant it writes it, then idempotently SET `camera = clean (+) head`. Safe at multiple sites per frame. See §6B for the change-detected implementation.

8D. Redirect-the-copy — never touch the game's memory

RDR2's elegant answer to a stack-resident input POV. Rather than rotating the game's own buffer (which lived on its stack, and corrupted it over long sessions → freeze/crash):

```
CaveRedirect(rdi):
   if (!enabled || !readable(rdi)) return rdi;          // hand back the original, untouched
   if (!Ortho(matrix at rdi)) return rdi;               // not a clean camera -> leave alone
   idx = g_bufIdx; g_bufIdx = (g_bufIdx+1) & (kNB-1);   // 8 heap buffers, round-robin
   buf = g_buf[idx];
   memcpy(buf, rdi, 0x100);      // COPY the whole input struct (pointers preserved)
   RotateBuffer(buf);            // rotate the COPY only
   return buf;                   // game reads our rotated copy; its stack is untouched

; in the cave:  mov rcx, rdi ; call CaveRedirect ; mov rdi, rax   <- redirect the register
```

Because `buf` is a fresh copy of the game's current input each call, it **is** already the clean value — no change-detection, no compounding possible, by construction. Elegant and worth reusing wherever the hook gives you a pointer the game is about to read.

8E. Per-pointer slot tables — mandatory when several cameras share a hook

Multiple camera objects (main / cull / shadow / reflection / cubemap) routinely flow through one write site. A single global clean base **thrashes** between them and corrupts both (§7B failure 3). Key the state by object pointer:

```
struct Slot { uintptr_t key; float clean[16]; float lastW[16]; float cleanP[3]; float lastP[3]; bool have; };
static Slot g_slots[16];
static Slot* slotFor(uintptr_t key){
    for(auto&s : g_slots) if(s.key == key) return &s;
    Slot* pick = &g_slots[(key>>6) % 16];   // hash-index reuse
    pick->key = key; pick->have = false; return pick;
}
```

16 slots is the library-wide standard (RDR2, NieR Automata, Ni no Kuni, DA2). Each camera keeps its own clean base so rotations never compound across them, while the head pose advances once per frame so every object that frame sees the same rotation. **Ni no Kuni also added** `MaxOrbitDistance` to optionally leave the far aux/reflection camera untouched entirely — a cheaper alternative when the aux camera is identifiable by geometry.

9. Version-agnostic camera location

Ordered most → least robust. All are INI-driven so a patch usually needs only an INI edit.

1. **Signature scan with wildcards.** INI carries `AOB` (`??` = wildcard), a BP/steal offset, and the struct offsets. Scan executable memory; hook `match + offset`. **The default for modern mods.**
2. **Pointer-chain resolve.** An AOB finds a global camera pointer; dereference a fixed chain.

```
Dead Space:  A1 ?? ?? ?? ?? 85 C0 75 ?? 53   -> camPtr = *(match+1) -> cam = *camPtr -> matrix @ +0x08
AC Unity/Syndicate:  manager -> *(*(manager+0x40)) -> transform = camObj + 0x50
P5R:         global (pattern-scanned) -> cam = *(*(global)+0x10) -> view 4x4 @ cam+0x08, proj @ cam+0x60
Deus Ex HR:  CameraManager = *(base+0x187CEC0) -> *(mgr+0x30) -> view @ cam+0x80
Quantum Break: cam_ptr global = match + 7 + int32@(match+3)   (RIP-relative resolve)
```

3. **Runtime auto-offset discovery.** Capture the writer's destination; if `dest − cam < 0x2000` it's inside the camera → derive the field offset live (Batman Origins). **Survives struct-layout shifts.**
4. **RTTI vtable resolve.** GTA IV resolves `CCamFinal`'s vtable by RTTI rather than by address — version-agnostic against a class whose layout is stable but whose address is not.
5. **Runtime behavioural discovery.** The strongest and most portable: no signature at all. GTA V's finder scans the heap for orthonormal matrices that *move as you look around* and sit next to a perspective projection, then locks the one matching `GET_FINAL_RENDERED_CAM_COORD` (falling back to the strongest differential mover). Pure runtime behaviour — survives any patch.
6. **Static address fallback.** Only when no signature can be formed. **Always behind** `UseAOB=0` so it's opt-in.

Packing / encryption and what it does to scanning

Protection	Titles	Consequence
None	P4G, Ni no Kuni (Steam DRM `.bind` stub only), Alan Wake Rem. (`.text` entropy 6.50)	AOB scan resolves at load. Static analysis of the exe works.
Denuvo	AC Origins	**Bans debug registers** → must use 5A, not 5B. But the camera path is *not* encrypted — all four IGCS AOBs are present and unique in the on-disk `.xcode`.
SteamStub	MGS5	Packed on disk. **Scan the decrypted code at runtime after a delay.** Keep the pattern register-agnostic.
Arxan	GTA V Enhanced	`.text` encrypted on disk (entropy 7.99), decrypted per-page at runtime. **Static scans see nothing.** Debug registers are lethal. Guard pages are lethal. **Code patches ARE viable** if minimal and verified. The only viable artifact is a decrypted runtime module dump (entropy 6.63 — disassemble offline).

Retry the scan; don't scan once. The camera write site may not exist until gameplay. The standard is a retry loop — RDR2: `for(int t=0; t<600 && !site; ++t){ site = Scan(...); if(!site) Sleep(100); }` (60 s); Guardians retries at ~250 ms cadence until the copy site is present.

Verify before you patch. GTA V's install checks the AOB **and** the exact entry bytes before patching, and **aborts safely** (logs, no patch) if anything differs — so a game update degrades to "no hook" rather than a crash. Adopt this everywhere.

10. The full-6DOF wobble self-test

Ships enabled so you can confirm the AOB hooks the right camera **on the desktop, with no OpenTrack running**. If the whole view pans by itself ~12–20°, the hook is correct; then set `WobbleTest=0` for real tracking.

10A. Per-axis toggle — the minimum standard

```
if(cfg.testWobble){
    float w = sinf(GetTickCount()*0.001f * 1.3f) * cfg.wobbleDeg;   // degrees
    float yaw=0, pitch=0, roll=0;
    if(cfg.wobbleAxis==0) yaw=w; else if(cfg.wobbleAxis==1) pitch=w; else roll=w;
    // position zeroed during wobble so you confirm LOOK before LEAN
    return;
}
```

`WobbleAxis` (0=yaw, 1=pitch, 2=roll) verifies each rotation axis independently. The log prints `[WOBBLE: whole view should pan = AOB OK]`.

10B. Full-6DOF named sweep — the standard going forward

This is the required default for every new mod. It dwells ~2.5 s on each of the six DOF in turn and names the live axis in the log, so a desktop run proves rotation **and** lean on every axis before a tracker is ever connected. Already implemented in RDR2 (`WobbleMode=1`) and Dead Space — promote it, don't rebuild it.

```
if(g.testWobble){
    double t = GetTickCount64()/1000.0;
    float s = (float)sin(TAU * g.wobbleHz * t);
    if(g.testMode == 1){                          // 6-axis sweep
        int a = ((int)(t/2.5)) % 6;  g_testAxis = a;
        float rv = g.wobbleDeg * s, pv = g.wobblePos * s;
        switch(a){ case 0: hy=rv; break;  case 1: hp=rv; break;  case 2: hr=rv; break;
                   case 3: lx=pv; break;  case 4: ly=pv; break;  case 5: lz=pv; break; }
    } else {                                      // single axis
        float w = g.wobbleDeg * s;  g_testAxis = g.wobbleAxis;
        if(g.wobbleAxis==0) hy=w; else if(g.wobbleAxis==1) hp=w; else hr=w;
    }
}
static const char* kAxisName[6] = {"YAW (look L/R)","PITCH (look U/D)","ROLL (tilt)",
                                   "POS X (lean L/R)","POS Y (up/down)","POS Z (in/out)"};
// log on transition:
if(g.testWobble && g.testMode==1 && g_testAxis != lastAxis){
    lastAxis = g_testAxis; if(g_testAxis >= 0) Log("wobble -> %s", kAxisName[g_testAxis]); }
```

Defaults: `WobbleDeg=15` (rotation amplitude), `WobblePos=0.3` (lean amplitude, metres), `WobbleHz=0.5`, dwell 2.5 s.

Reading the result

Observation	Meaning / action
Whole view pans smoothly	AOB is on the right camera. Proceed. Try each axis.
Nothing moves	Wrong object, wrong offset, or the write doesn't reach the screen. Go to §13.
Only part of the scene moves	You're on a cull/shadow/reflection camera, not the render camera. Add the address-select capture cave.
View moves but geometry shears	Wrong matrix convention — transpose, or stride (§7B).
Moves then snaps back	Persistent camera + absolute writes. Wrong drive model (§8A).
Moves, but the game fights it	Distance/collision system reacting (§8D) — reduce `WorldUnitsPerMetre`.

11. FOV options & cutscene FOV

Every FOV approach in the library

Approach	How	Used by
Offset-add	`fov += Offset°` (negative zooms in)	Batman, Alan Wake Remastered, Control
Scale / absolute	`FovScale>1` widens; `FovDegrees>0` pins an absolute vertical FOV	Dead Space, DXMD (with Min clamp), RDR2
Encoding-aware	radians vs degrees; `fovBaseDeg`	MGR (base 66°), Yakuza Kiwami 2 (radians), RDR2 (`IsRadians`)
tan-half encoding	The field is `tan(vfov/2)`, not an angle. Write `tan(want/2)`. Or a 1.0-centred factor.	Alan Wake Remastered (`Encoding=2`, base 90° vfov, field 1.0 == 100%)
Current + target dual-field	Read the game's target FOV, write the current — survives the engine's own FOV lerp	NieR Automata (`fovOff=0x138`, `fovTgtOff=0x188`)
Projection-consistent	Rescale `proj[0]` / `proj[5]` and the `fovY` field **together** so cull frustum and render match	DA:Origins

Projection-consistent scaling (correct when you own the matrix)

```
float s = fovScale;
if(fovDegrees > 0.5f){
    float want = fovDegrees * D2R, bt = tanf(baseFov * 0.5f);
    if(bt > 1e-4f) s = tanf(want * 0.5f) / bt;
}
if(fabsf(s - 1) > 1e-4f){
    P[0] = baseP[0] / s;
    P[5] = baseP[5] / s;
    F[0] = 2 * atanf(tanf(baseFov * 0.5f) * s);
} else { memcpy(P, baseP, 64); F[0] = baseFov[0]; }
```

The radians-aware generic form (RDR2)

```
void ApplyFov(uintptr_t pov){
    if(!g.fovEnabled || !g.fovOff || !Writable(pov + g.fovOff, 4)) return;
    float v = *(float*)(pov + g.fovOff); if(v != v) return;      // NaN gate
    bool rad = g.fovIsRadians;
    float deg = rad ? v / D2R : v;
    float want = (g.fovDegrees > 0.5f) ? g.fovDegrees : deg * g.fovScale;
    want = Clamp(want, g.fovClampLo, g.fovClampHi);              // 5 .. 130
    *(float*)(pov + g.fovOff) = rad ? want * D2R : want;
}
```

`FovFromZ` couples FOV to head lean in/out ("lean in to zoom"): `s *= (1 + clamp(sz*fovFromZ, fovClamp)*D2R)`.

Cutscene FOV — four detectors UPDATED

Cutscenes typically run a **lower** FOV than gameplay. The goal: detect the state and **leave the cutscene's own FOV alone** (`FovDegreesCutscene=0`), optionally easing tracking authority (`CutsceneTrackScale`). Four independent signals exist in the library; pick by what the engine gives you, and **always debounce**.

#	Detector	Mechanism	Precedent
1	**FOV-magnitude baseline**	Track a slow gameplay-FOV EMA baseline (while not in a cutscene); call "cutscene" when native FOV drops below `baseline − CutsceneDropDeg` with a debounce.	P5R detector, inverted
2	**FOV range window**	Scalar in `[lo, hi]` means the special state. Simplest; needs known constants.	P5R (`CutsceneFovLow/High`, `FovBattleLow/High`)
3	**Hook staleness** NEW	One hook site **freezes** during cutscenes while another keeps firing. If site A has been idle > N ms, you're in a cutscene.	Control: `renderMainView` idle > 120 ms == cutscene (`rmv` freezes; `bom` does not)
4	**Sentinel scan**	Some states write a huge/inf/NaN flag into a struct slot; find it anywhere in the object (`fabsf(fv) >= 1e18f || fv != fv`).	P5R
5	**Motion detector**	Smoothed frame-to-frame camera-rotation delta with an enter/exit state machine (enter after sustained N ms above `hi`, exit after calm N ms below `lo`).	P5R (**off by default** — P5R's field camera pans as hard as battle, so motion is unreliable there)
6	**Native API**	Ask the game. `IS_CUTSCENE_PLAYING` / `IS_CUTSCENE_ACTIVE`.	GTA V

Hysteresis is mandatory

All detectors must debounce or they flip-flop at the boundary and strobe the FOV. Library defaults: Control uses `fovHysteresisMs = 350` ("cutscene state must hold this long before the FOV add switches — kills flicker") and `rmvStaleMs = 120`. P5R uses `battleMotionMs = 350` (enter on sustained motion, exit when calm). Recommended new-mod defaults: `CutsceneDropDeg=8`, `CutsceneDebounceMs=250`.

Self-disable when the FOV field is invalid

> The detector **must** self-disable where the field isn't a plausible FOV. DA2's `+0x140` reads zero → ships `NoFov=1`, and it uses a VIEW/WORLD orthonormal-pair test to pick the camera instead. DA:Origins' `+0x140` is valid (gated 0.3–2.5 rad) so its detector runs. Gate on a plausible range before trusting the signal.

The general lesson from Control

Control's `fovAddCutscene` is marked **DEPRECATED** in the shipped source: "cutscene FOV split removed — one FOV value is used everywhere". Worth knowing before re-adding complexity: on that title the split wasn't worth it. The reason to keep a cutscene FOV path is to *leave the cutscene's FOV alone*, not to impose a second value.

12. Smoothing (the "no jitter" filter)

EMA — legacy (`SmoothMode=0`)

`sm += a*(raw − sm)` . Make it frame-rate-independent with `a = 1 − exp(−dt/τ)` ; larger `Smoothing` → larger τ → smoother (a touch more latency). Kills most shimmer, but its **velocity is discontinuous** — which is what you feel as micro-shake.

Critically-damped spring — the default (`SmoothMode=1`)

> **The true no-shake upgrade.** Continuous velocity → tracker noise comes out as sub-pixel drift instead of shimmer. Game-Programming-Gems `SmoothCD` .
> **Run it on all six channels.** `SmoothHz ≈ 8` is the no-jitter default (higher = snappier / less smooth).

```
void SmoothCD(float& val, float& vel, float target, float dt, float omega){  // omega = 2*pi*SmoothHz
    float x = omega*dt;
    float e = 1.0f / (1.0f + x + 0.48f*x*x + 0.235f*x*x*x);   // fast exp(-x) approximation
    float change = val - target;
    float temp   = (vel + omega*change) * dt;
    vel = (vel - omega*temp) * e;
    val = target + (change + temp) * e;
}
```

```
// worker loop, all six channels
uint64_t now = GetTickCount64();
float dt = lastT ? (now-lastT)/1000.0f : 0.008f;  lastT = now;
if(dt < 0.0005f) dt = 0.0005f;  if(dt > 0.1f) dt = 0.1f;    // clamp: alt-tab / hitch guard
if(!sInit){ sY=hy; sP=hp; sR=hr; sX=lx; sYy=ly; sZ=lz; sInit=true; }   // seed, don't ramp from 0

if(g.smoothMode == 1){
    float w = (float)TAU * g.smoothHz;
    SmoothCD(sY,vY,hy,dt,w); SmoothCD(sP,vP,hp,dt,w); SmoothCD(sR,vR,hr,dt,w);
    SmoothCD(sX,vX,lx,dt,w); SmoothCD(sYy,vYy,ly,dt,w); SmoothCD(sZ,vZ,lz,dt,w);
} else {
    float a = g.smoothEMA;
    sY+=a*(hy-sY); sP+=a*(hp-sP); sR+=a*(hr-sR); sX+=a*(lx-sX); sYy+=a*(ly-sYy); sZ+=a*(lz-sZ);
}
```

> **Three details that matter.** (1) **Clamp dt** to [0.5 ms, 100 ms] — an alt-tab or hitch otherwise makes the spring explode. (2) **Seed on first run** (`sInit`) instead of ramping from zero, or the view swings in at startup. (3) The smoothed pose is computed **once per frame on the worker** and read lock-free by the cave — the handler must never do I/O or allocate.

Other jitter sources smoothing will NOT fix

- **Resting bias** — the tracker's neutral is rarely exactly zero (NieR measured +0.03 m on Y). With `WorldUnitsPerMetre=10` that constantly lifts the eye. Fix with a **lean deadzone** (`LeanDeadzone=0.02` m) and recenter while seated naturally.
- **Engine fighting the write** — the distance/collision oscillation (§8D). Smoothing makes it prettier, not gone.
- **Double-apply** — two hooks (or a hook + a VEH) both applying → judder. See the AC Unity fix (§7B).
- **Glitch spikes** — Crimson's model: reject jumps > 0.12 m, deadzone < 0.012 m, clamp ±0.35 m.
- **Baseline recenter** — Batman AK captures a *smoothed* baseline on HOME rather than a raw snapshot, so recenter is itself jitter-free.

13. Failure-mode catalogue

The diagnostic table this project earned the hard way. Match the symptom, read the cause.

Symptom	Cause	Fix
Perfect diagnostics — values applied, orthonormal maintained — but nothing on screen	**You are editing the wrong copy.** Logic/telemetry camera ≠ render camera. Or an upstream copy the engine recomputes.	Find the render camera by behaviour (GTA V's finder), by RE of a working reference DLL (Crimson), or by find-what-writes on the located object. **The single most expensive trap in the project.**
Write lands (log proves memory changed) but the view doesn't move — even with a huge constant	The buffer isn't read for render. Crimson's "PageDown constant-nudge" probe: a large constant on `camBase+0xC8` and on the "final camera" object moved **nothing**.	**Build the constant-nudge probe first** — it is decisive and cheap. Then target the vtable-returned transform, or saturate the read sites.
Async writes never stick	The game rewrites the frame every game-frame; your worker-thread write is overwritten before anything consumes it.	**Hook in-thread.** Async can never win that race. (GTA V Mode 9's "timing wall".)
Yaw and roll stick, PITCH does not	The third-person controller **re-derives** pitch downstream of the camera copy you hooked. Proven in AW2 logs: `wroteFwdY 0.39 -> gameLeftFwdY 0.000`.	Hook a **later, final** stage — AW2 uses the engine's view+FOV setter (`SetViewAndFov`).
Head-look never persists ("fixed camera"); lean barely noticeable	**Reset camera + incremental model.** The per-frame reset discards accumulated rotation; the follow-cam absorbs incremental lean.	Absolute-on-live (§8B).
Camera pins, then snaps back; right stick starved	**Persistent camera + absolute writes with change-detection.**	Incremental delta, or absolute-on-live if it's actually a reset camera.
Camera spins and tilts; eye→target distance collapses (4.00 → 0.70 → 0.39)	**Two cameras sharing one global change-detection base** (gameplay + a fixed reflection rig at distance exactly 50). Both accumulate.	Per-pointer slot table (§8E). Optionally `MaxOrbitDistance` to skip the aux camera.
In/out oscillation when leaning hard	The engine's **camera-distance/collision system** reacting to the displaced eye (`+0x50` spiked to 2500 vs `+0x134` at 4.47).	Lower `WorldUnitsPerMetre` (10 → 1.0); add `LeanDeadzone` .
Low fps + judder	**Double-apply:** a VEH data-breakpoint firing an exception storm on top of a working function hook.	Make the HW-BP path a **fallback only** (AC Unity fix).
Crash on startup; log shows hook installed but no camera captured	The engine **reuses one address for different structure types**. NieR: usually a look-at camera (w == 0), periodically a frustum/bounds rig (w = 2.0/1.0). No previous frame exists at startup so the scene-cut guard can't fire.	**Validity gate** (eye.w & target.w ≈ 0 AND `+0x134` == |eye−target|) + **warm-up** (8 stable frames before the first write).
Crash on loading a save / scene cut	The engine rebuilds the camera; eye/target teleport, distance jumps, w-components flip. Writing into a half-rebuilt camera crashes. The object may even be **partially unmapped** — a read can race the free and fault.	**Discontinuity guard**: detect eye jump > 25 units, or distance changing >1.8× / <0.55×, or distance outside [0.05, 1e5] → stop writing ~90 frames, re-sync bases. Plus a **scoped VEH guard** with `longjmp` around our own memory ops.
Writing a field crashes inside a call	The field is a **pointer**, not floats. NieR `+0x60` is a pointer that `FUNCTION A` dereferences.	Keep a per-game **do-not-write list**. NieR: never write `+0x60` (pointer) or `+0x130` (live zoom the engine reads).
Nothing injects at all	**Wrong proxy name.** Alan Wake Remastered imports *neither* `dinput8.dll` *nor* `version.dll` — an ASI loader under those names never loads.	Check the exe's imports. AWR imports `dbghelp.dll` (only `MiniDumpWriteDump`) and `dxgi.dll` (only `CreateDXGIFactory`) → ship a `dbghelp.dll` proxy. Ni no Kuni imports dxgi/d3d11 but **not** dinput8.
Log: "cave out of rel32 range (~-5GB)"	`AllocNear` blind-allocating across ±1 GB, failing on reserved pages, falling back to far memory.	Walk `MEM_FREE` regions across ±2 GB; add the 14-byte `FF 25` absolute-jump fallback (§6A).
Matrix checks fail; nothing written	**Wrong stride.** Assumed 0x10; Northlight packs a tight 3×3 at stride `0x0C` , so "row2" starts at +0x20 and collides with the position (length ~1400).	`RotStride=12` . Verify all three rows read as unit length.
Rotation is wrong / view shears	Treating an Euler triplet as a quaternion (Witcher 3 +0x70), or wrong transpose.	Dump the fields. Degrees look like `323.6 / 280.5 / 17.5 / 1.0` . Quaternion components are in [−1,1] with unit length.
Yaw or roll skews diagonally	All three axes applied on the same multiply side while the camera is pitched.	**Split-axis rotation**: `A' = Ry * A * (Rx*Rz)` (§7B).
Roll simply won't apply	It's a **look-at rig** — no roll DOF exists in memory.	Stop trying. Roll requires family 5E (§8D).
Writing the euler fields does nothing	They're consumed **upstream** of your hook.	Trace the finalize function to find what's actually read downstream.
View starts rotated / off-centre	Recenter baseline captured from the all-zero default instead of the first real packet.	Recenter from the **first REAL packet**.
Game crashes when the mod arms	HW breakpoint armed during loading; or debug registers on Denuvo/Arxan; or a guard page on Arxan.	Gameplay-detector before arming; use 5A on Denuvo/Arxan; use `PAGE_READONLY` + single-step, never `PAGE_GUARD` .
Game freezes/crashes after a long session	Rotating the game's own **stack-resident** buffer corrupts its stack over time.	**Redirect-the-copy** (§8D) — never touch the game's memory.
Mod loads twice (two banners)	The DLL is present under two proxy names; a second receiver competes for UDP 4242.	Single-instance mutex; ship one proxy.

The find-what-writes technique, safely

> **Never use `PAGE_GUARD` .** It fires on **every** access (reads included) and re-arms inside the handler — an exception storm. On Arxan it crashes the game outright. **Guard pages are detected and lethal.**

```
The safe method (GTA V's breakthrough):
  1. Set the located camera's page to PAGE_READONLY   -> only WRITES fault (far fewer exceptions)
  2. In a VEH: on a write into the camera matrix, record the faulting RIP,
     make the page writable, set the single-step trap flag, continue
  3. On the single-step, re-protect the page

Pointed at a camera already located by float-differential, this ran ~6 s while the player
moved the view, did NOT crash Arxan, and logged the exact writers:
  GTA5_Enhanced.exe+0x1ae16a   hits=535   movsd [rcx+0x40],xmm1   (POSITION)
  GTA5_Enhanced.exe+0x30e76    hits=535   movss [rcx+0x10],xmm8   (ROTATION)
```

Why it was needed: static signature scans **cannot see indirect writers**. GTA V's per-frame writer computed its destination via register/`memcpy`, invisible to every form of rip-relative SSE store scan, `lea reg,[rip+frame]` scan, and global-pointer scan. Only the one-time *init* writer was findable statically. **Capture the writer at runtime on the address you already located.**

The payoff: both writers turned out to be inside `0x1ae150` — a **generic "copy a source pov into a camera struct"** function with 61 callers. One caller passes the logic frame; another passes the render camera. The earlier hook of that same function had been structurally correct all along — it was merely **gated to the wrong destination**. Hooking it and gating on `dest == g_hObj − 0x10` gives head-tracking in **gameplay and cutscenes from one hook**.

PART III — THE CAMERA-DATA CORPUS

14. Corpus overview & what it is

A research archive of community camera work — memory-scan recipes, freecam tables, injector-class tool binaries and their source constants, and GPU/shader profiles. **It is not code to reuse; it is a lookup table.** Before writing a line for a new game, look it up here. Very often the camera is already solved.

This document records only the data — signatures, offsets, vtable/slot indices, constants and conventions. Tool names, file names, authors, counts and provenance are irrelevant to building a mod and are omitted throughout. What matters is *which kind of artifact answers which question*:

Kind of artifact	What it gives you
Memory-scan tables	XML + assembler scripts. Each carries **AOB signatures**, the injection point, the stolen bytes (in the disable block), and the struct offsets used by the patch code. The single largest source of AOBs.
Injector-class tool binaries	Disassemble to recover write-site AOBs and offsets. They share a fixed **AOB label vocabulary** (§15).
Injector-class constant headers	**The single richest artifact.** Per-game struct offsets, intercept RVAs, **steal lengths** and defaults in plain text — no disassembly needed. Heavily duplicated; dedupe by game before drawing any conclusion.
GPU / shader profiles	Per-API matrix handling: transpose, validation, roll permission, reprojection, constant-buffer slot search, map type. The **family-6E targeting data**.
CPU scan profiles	Full memory-scan recipes: which regions, block layout, per-axis offsets, value ranges, apply codes, majority vote.
Flattened scan tables	The scan profiles as rows: game, arch, API, exe, FOV expression, block, scan type, axis, byte offset, apply type, min/max, units.
Head-tracking tuning tables	Per-game tuning: sensitivity, roll enable, inverts, positional multiplier, crouch/jump thresholds. The basis of §19.
Shader matrix locators	Per-game: API, matrix constant **name**, whether a CPU scan exists, shader hash present, CB slot present.
Constant-buffer matrix locations	CB slot + buffer size + matrix byte offset + size + inverse flag. Direct input for a 6E build.

Why the names and counts are gone. An artifact's filename tells you nothing you can hook with, and a raw file count is not a finding — the archives are duplicated between 4× and 8×, so the only figures worth quoting are the **deduplicated** ones, which appear as findings in §15–§19 where they are actually derived. Cite the data, never the container.

15. The universal engine signatures HIGHEST VALUE

Of 882 unique AOBs across 596 tables, **38 are shared by 3+ games**. Those shared signatures are the payoff: for any game in a covered engine family, **the camera is already found**. This is the corpus's single most useful output.

15.1 The dominant modern-engine cluster

41 titles share one camera-write signature; 32 share one camera-address signature. These are `*-Win64-Shipping.exe / *-WinGDK-Shipping.exe` titles — i.e. the dominant commercial engine's shipping build. **52 such tables are present in the corpus.** If a new game ships under that exe naming convention, you already have its camera.

Hook 1 — camera-address capture (32 tables)

```
AOB:  48 8D 91 ?? 1? 00 00 48 8B CB E8 ?? ?? ?? ?? 48 8B C3 48 83 C4 20 5B C3
      (a longer variant prefixes: 48 83 EC 20 48 8B DA )

decodes to:   lea rdx,[rcx+0x1E40]      ; rcx = the camera-manager object
              mov rcx,rbx
              call ...                  ; -> the POV getter

patch:        lea rdx,[rcx+0x1E40]
              mov [cam], rdx            ; CAPTURE the POV address
              jmp return
```

Why the wildcard is `?? 1?`. The POV offset inside the manager object **moves between engine versions** (0x1E40, 0x1?40, ...). Wildcarding the two nibbles that vary makes one signature match dozens of builds and years of versions. **This is the cleanest version-agnostic idiom in the entire corpus** — wildcard the operand that drifts, keep the opcodes exact.

Hook 2 — the generic POV copy (41 tables)

```
AOB (short, unique):  F2 0F 11 01 48 8B DA 8B 42 08 89
full stolen block (35 bytes, from the [DISABLE] restore):
  F2 0F 11 01               movsd  [rcx],xmm0        ; POV.Location.X, .Y   (8 bytes)
  48 8B DA                  mov    rbx,rdx
  8B 42 08 / 89 41 08       mov    eax,[rdx+08] ; mov [rcx+08],eax    ; Location.Z
  F2 0F 10 42 0C            movsd  xmm0,[rdx+0C]
  F2 0F 11 41 0C            movsd  [rcx+0C],xmm0     ; Rotation.Pitch, .Yaw
  8B 42 14 / 89 41 14       mov    eax,[rdx+14] ; mov [rcx+14],eax    ; Rotation.Roll
  8B 42 18 / 89 41 18       mov    eax,[rdx+18] ; mov [rcx+18],eax    ; FOV

  rcx = DESTINATION pov      rdx = SOURCE pov
```

This is a "copy a source POV into a camera struct" function — structurally identical to the flagship RAGE mod's `0x1ae150`, which had 61 callers. Same shape, different engine. The tables gate it exactly as the flagship does:

```
code:
  mov rbx,rdx
  push rax
  mov rax, cam
  mov rax, [rax]
  cmp rcx, rax          ; <-- THE GATE: is this write targeting OUR camera?
  pop rax
  je  return            ;     yes -> skip the engine's copy (freecam takes over)
  ...run the original copy...   ; no  -> let every other camera through untouched
```

The additive difference. A freecam **skips** the copy when the gate matches (it owns the camera). We **do not skip** — we let the copy run, then add the head pose to the result (or edit the source POV before it). Same hook, same gate, opposite branch. This is the single most important adaptation when converting corpus data into a head-tracking mod.

The POV struct (the most common camera layout in existence)

Offset	Field	Type / notes
`+0x00`	Location.X	3 floats, world units
`+0x04`	Location.Y	
`+0x08`	Location.Z	
`+0x0C`	Rotation.Pitch	**3 floats, DEGREES** in modern versions. (In the older engine generation these are **ints**, 65536 = 360°.)
`+0x10`	Rotation.Yaw	
`+0x14`	Rotation.Roll	
`+0x18`	FOV	float, degrees

Independently corroborated by the constant-header corpus: the **rotation-minus-position delta of `+0xC` is the single most common camera layout** across the deduped game set (§16).

15.2 The managed-runtime cluster (8+ tables)

Titles built on a common managed runtime hook the runtime DLL, not the exe — the signature is engine-wide, so it is identical across every game on that runtime. **15 tables in the corpus.** Same capture→gate→edit shape:

```
position write:   48 03 50 18 0F 10 02 0F C2 C2 04 0F 50 C0 0F 11 12 A8 07 0F 84 ?? ?? ?? ?? 4C 8B 15 ...
                  add rdx,[rax+18] ; movups xmm0,[rdx] ; cmpps ; movmskps ; movups [rdx],xmm2
                  -> gate:  cmp rdx,[cam]   ; transform POSITION written to [rdx]

rotation write:   0F 56 DC 0F C2 C3 04 0F 50 C0 41 0F 11 5A 10          (8 tables)
                  orps xmm3,xmm4 ; cmpps ; movmskps ; movups [r10+0x10],xmm3
                  -> gate:  cmp r10,[cam]   ; transform ROTATION (quaternion) at [r10+0x10]
```

Struct: position at `[obj]`, rotation quaternion at `[obj+0x10]` — the **+0x10 delta** layout (§16). Two hooks because position and rotation are written by different routines; both gate on the same captured object.

15.3 Controller / input signatures (23 + 14 + 12 tables)

Present in most freecam tables to *steal* the gamepad. **We do not need them** — head-tracking never takes input. Recorded only so they are recognised and skipped when reading a table:

```
48 8B C4 57 48 83 EC 60 80 B9 ?? ?? ?? ?? 00 48 8B F9 0F 29    (23 tables)
48 8B C4 57 48 83 EC ?? 80 B9 ?? ?? ?? ?? 00 48 8B F9 0F 29    (14 tables — steal-size wildcarded)
53 33 DB 55 0F B7 6C 24 10                                      (18 tables — XInput state)
40 53 48 83 EC 60 48 8B 05 ?? ?? ?? ?? 48 8D 54 24 20          (6 tables — gamepad read)
```

15.4 The table symbol vocabulary — read any table in seconds

The corpus uses a stable naming convention. Recognising it tells you what each script is for without reading the assembler:

Symbol	Count	Meaning & value to us
`instr_read_cam`	137	**Camera-address capture.** Hook 1 of the universal method. **Always start here.**
`instr_write_cam`	134	**Camera struct write.** Hook 2. The primary target.
`instr_write_cam_1..4`	69	Multiple write sites — the engine writes the camera at several places. **Saturate them all** or find the last one in the frame.
`instr_write_vm` / `instr_read_vm`	104	**VIEW MATRIX** read/write. Signals a matrix-convention game (§7B), not a quat/POV one.
`instr_write_vm_1..3`	40	Multiple view-matrix writes.
`instr_write_quat`	12	Quaternion convention (§7A).
`instr_write_lookat`	7	**Look-at rig** (§7D) — expect no roll DOF.
`instr_write_fov`	13	FOV write site — also the **reliable gameplay-camera identifier**.
`instr_read_cam_struct`	6	Struct-address resolve.
`instr xinputgetstate`, `instr_read_controller`, `instr_disable_controller`	190+	**Input theft — ignore.** Not needed for head-tracking.

How to read a table in 60 seconds. (1) Find `aobscanmodule(...)` → the AOB and target module. (2) Read the `assert(...)` or the `[DISABLE] db ...` block → **the full original bytes**, which give you the steal length and let you decode the struct offsets. (3) Read the `code:` block → the register holding the camera and the gate. (4) The trailing comment block gives the **injection point RVA** and surrounding disassembly.

15.5 The injector-class label vocabulary

Tool binaries store their AOB labels as plain strings — `strings` the DLL and you get the hook inventory instantly. Across 19 tools, 219 distinct labels; the camera-relevant ones:

Label	Freq	Meaning
`AOB_CAMERA_ADDRESS_INTERCEPT`	**15/19**	**Hook 1 — the universal first hook.** Present in 79% of tools.
`AOB_CAMERA_WRITE1..4_INTERCEPT`	18	Hook 2 — **numbered = saturation** across multiple write sites.
`AOB_CAMERA_FOV_WRITE_INTERCEPT`	2	FOV write — camera identifier.
`AOB CAMERA DATA WRITE INTERCEPT`, `AOB_CAMERA_MANAGER_INTERCEPT`, `AOB_CAMERA_WRITE_LOCATION`	4	Variants of the same two roles.
`AOB CAMERA NEARPLANE DITHER LOCATION`, `AOB_CAMERA_CLIP_CHECK_LOCATION`, `AOB_LOOKAT_CLIP_CHECK_LOCATION`	4	Near-plane / clip disable. **Occasionally useful** — heavy lean can clip the camera through geometry.
`AOB_AR_CORRECTION_REMOVAL_LOCATION`, `AOB CUSTOM ASPECT RATIOS ADDRESS`, `AOB_ASPECT_RATIO_CLAMPER_LOCATION`	5	Aspect-ratio clamps — relevant if FOV work fights the engine.
HUD / TOD / timestop / LOD / time-dilation / pause labels	~25	**Not camera. Ignore.**

The recovery method. Extract the label strings; the byte pattern is stored **immediately before the label in** `.rdata`, so pairing is mechanical. Then locate the tool's own interceptor stub by searching its `.text` for the **re-embedded stolen bytes**, and disassemble it — the stub shows the exact offsets written and the gate used. If an open source tree exists for the tool, prefer it: the constants are in plain text (§15.6).

15.6 The constant headers — 49 games, no disassembly required

Per-game headers give the whole camera model in plain text — **including the steal length, for free**. The schema:

```
#define the game name field                                  "..."
#define CAMERA_ADDRESS_INTERCEPT_START_OFFSET      0xCAF5     ; hook 1 site (RVA)
#define CAMERA_ADDRESS_INTERCEPT_CONTINUE_OFFSET   0xCAFD     ; resume
        ;;  STEAL LENGTH = CONTINUE - START = 8 bytes.   <-- free, exact, no disassembly
#define CAMERA_WRITE_INTERCEPT1_START_OFFSET       0x71EFC    ; hook 2 site
#define CAMERA_WRITE_INTERCEPT1_CONTINUE_OFFSET    0x71F08
#define LOOK_QUATERNION_IN_CAMERA_STRUCT_OFFSET    -0xC8      ; orientation  (may be NEGATIVE)
#define CAMERA_COORDS_IN_CAMERA_STRUCT_OFFSET      -0xD8      ; position
#define FOV_IN_CAMERA_STRUCT_OFFSET                0x8
#define INITIAL_PITCH_RADIANS / _YAW_ / _ROLL_     0.0f
#define CONTROLLER_Y_INVERT                        true
#define DEFAULT_FOV_DEGREES                        ...        ; the game's native FOV — useful for §11
```

Key	Freq	Use
`CAMERA_COORDS_IN_CAMERA_STRUCT_OFFSET`	114*	Position offset.
`FOV_IN_STRUCT_OFFSET` / `FOV_IN_CAMERA_STRUCT_OFFSET`	108 / 81*	FOV offset.
`LOOK DATA IN CAMERA STRUCT_OFFSET` / `LOOK_QUATERNION_...` / `MATRIX_IN_STRUCT_OFFSET`	82*	Orientation offset. **The key name tells you the convention** (§7): `QUATERNION` → 7A, `MATRIX` → 7B, `LOOK_DATA` → inspect.
`CAMERA_ADDRESS_INTERCEPT_KEY`	147*	Hook 1 exists (74% of games).
`CAMERA_WRITE1_INTERCEPT_KEY` / `WRITE2`	75 / 65*	Hook 2; **65 games need ≥2 write sites**.
`DEFAULT_FOV_DEGREES`	87*	Native FOV — the baseline for absolute-FOV work.

* raw file counts across 200 files; divide by ~4 for unique games.

The steal length is free. `CONTINUE − START` gives the **exact** number of bytes to steal, already validated by a shipping tool — no disassembly, no instruction-length decoding, no guessing where an instruction boundary falls. Observed values span **6, 7, 8, 12, 16, 17, 21, 60, 87, 104 bytes**: there is **no "standard" steal length**, and the large ones (60–104) are cases where the hook must swallow an entire block to land on a safe boundary. This single subtraction removes the most error-prone step in building a code cave (§6A), and a wrong steal length is the **single most common cause of an instant crash on install** (§13).

Offsets are frequently NEGATIVE. The captured anchor is often mid-struct, so coords/quat sit *before* it (`-0xC8`, `-0xD8`, `-0x30`, `-0x78`). Never assume a positive offset from the captured register — and note the flattened scan CSV also carries negative byte offsets (e.g. yaw at `-24`).

15.7 The camera-write instruction vocabulary NEW

Classifying the leading opcode of all **539 camera-related AOBs** across the 345 unique tables answers a question that recurs on every new game: *what am I actually looking for when I do find-what-writes?*

Instruction class	Share	What it means for the hook
`MOVUPS/MOVAPS [mem],xmm`	**17.1%**	**vec4 write.** A whole position/quaternion/matrix row in one store. Implies a 16-byte-aligned layout → the **+0x10 / −0x10** struct family (§16).
`MOVSD [mem],xmm`	**12.6%**	**Two floats (or one double) at once.** The signature of a *tight* POV struct — this is exactly the engine-sibling cluster's `movsd [rcx],xmm0` writing Location.X+Y as one 8-byte store (§15.1). Implies the **+0xC** family.
`MOV reg,[mem]` / `MOV [mem],reg`	**20.0%**	Integer moves — copying floats via GP registers, common inside copy loops. Also the **UE-style int rotation** (65536 = 360°).
`LEA reg,[base+disp]`	**10.6%**	**The ADDRESS-capture signature.** Not a write at all — the engine computing the camera/POV pointer. **This is Hook 1 of the universal method** and the reason `48 8D 91` is the 3rd-most-common AOB prefix in the whole corpus.
`MOVSS [mem],xmm`	**9.8%**	**Single float write.** Per-component writes → expect **several** write sites (hence `WRITE1..4`).
`MOVUPS/MOVSS/MOVSD/MOVAPS xmm,[mem]` (loads)	**18.8%**	**READ sites.** Hooking a read is a legitimate and sometimes superior strategy — you edit the value *as the engine consumes it*, so nothing downstream can overwrite you (§6C).
`MOVDQA/MOVDQU [mem],xmm`	2.8%	Integer-SIMD move of float data — same meaning as MOVUPS.
`FLD` / `FSTP` (x87)	**3.9%**	**32-bit-era only.** If you see x87 you are in an old x86 title → family 6C, and **you must preserve FPU state** (`fnsave`/`frstor`) around your call (§13).
`CALL` (direct)	2.6%	Hooking the call rather than the write — the "generic copy function" pattern (§15.1).
`CALL/JMP [reg+disp]` (**vtable**)	**0.7%**	**Rare — and that is the finding.** See below.

TWO CONCLUSIONS THAT CHANGE HOW YOU START A NEW GAME

1 — Look for an SSE store to `[reg+disp]`. That is the camera, 42% of the time. MOVUPS + MOVSD + MOVSS + MOVAPS + MOVDQA stores together are **42.3%** of all camera hook sites. Add the SSE *loads* (18.8%) and **61% of every camera hook in the corpus is a single SSE instruction touching `[register + displacement]`**. The displacement in that instruction *is* the struct offset you are looking for, and the register *is* the camera pointer. When find-what-writes gives you a list, this is how you pick.

2 — Vtable dispatch is 0.7%. The camera is almost never behind a virtual call. Only 4 of 539 camera sites are indirect calls, and the indices are scattered (`+0x08`, `+0x10`, `+0x28`, `+0x30`, `+0x38`, `+0x40`, `+0x50`, `+0x68` — no consensus). **This retroactively explains the hardest title in the atlas.** Its render transform is reached by `call [vtable+0x18]` — and the corpus says that shape occurs in well under 1% of games. It was not a misunderstanding; it is a genuine outlier, and the weeks it cost were the cost of a rare case, not a wrong method. The corollary for planning: **if a game hides its camera behind a vtable, expect it to be hard, and budget for it.**

15.8 Convention declared by name NEW

The constant headers name the orientation key after the convention it holds. Across the 41 deduped games that declare one, that name alone tells you which §7 math to use — before you look at a single byte:

Constant key	Games	→ Camera math
`LOOK_QUATERNION_IN_CAMERA_STRUCT_OFFSET` `QUATERNION_IN_STRUCT_OFFSET`	**18** (44%)	**§7A quaternion.** The most common convention.
`LOOK_DATA_IN_CAMERA_STRUCT_OFFSET`	**14** (34%)	**Ambiguous by design — inspect.** "Look data" is the corpus's hedge when the block is a look-at rig, an Euler triple, or a basis. **Assume §7D look-at until disproven**, and expect no roll.
`MATRIX_IN_STRUCT_OFFSET`	**9** (22%)	**§7B matrix.** Check stride 0x0C vs 0x10 immediately.

This is a near-exact match to our own library's distribution across 51 mods — quaternion and look-at dominate, matrix trails. **The taxonomy in §7 is not an artefact of which games we happened to pick; it is the real shape of the problem.**

15.9 Where camera fields actually live NEW

Decoding every `[register + displacement]` operand inside the camera patch code of **310 tables** (stack frames excluded) gives a direct answer to "how big is a camera struct, and where do I look?"

Offset	Share	Cumulative	What sits here
`+0x08`	**14.6%**	14.6%	Location.Z (tight POV) · 3rd float of a vec3 · 2nd half of a vec2 store
`+0x10`	**12.3%**	26.9%	**Rotation/quaternion** in the +0x10 family · 2nd matrix row · position in the −0x10 family
`+0x18`	**11.7%**	38.7%	**FOV** (tight POV) · quat.w · row padding
`+0x0C`	**10.6%**	49.2%	**Rotation.Pitch** (tight POV) · translation in a 3×4 affine (4th column)
`+0x14`	7.9%	57.1%	Rotation.Roll (tight POV)
`+0x20`	7.1%	64.2%	3rd matrix row · focus/target vec4 · FOV in padded layouts
`+0x30`	6.3%	**70.5%**	**Position** in a 4×4 (row 3) · coords in the constant-header convention
`+0x04`	4.3%	**74.8%**	Location.Y
`+0x1C`, `+0x28`, `+0x40`, `+0x50`	2.1–3.1%	85.6%	Second blocks, mirrors, FOV in larger structs
everything else ≤ `+0x74`	—	**94.4%**	Tail fields: near/far, aspect, extra FOV, flags

THREE NUMBERS THAT SHAPE EVERY BRING-UP

70% of camera field accesses are within the first 0x30 bytes. 94% within 0x74. A camera struct is *small*. **Dump 0x80 bytes from the captured pointer and you have essentially the whole camera** — position, orientation, FOV, and the validators. There is no need to trawl kilobytes. This is why `DumpStruct=1` (§21.2) is worth having in every mod: one 0x80-byte hex+float dump next to a known head pose answers nearly every layout question in one frame.

The top 8 offsets are 75% of everything — and they are literally the POV struct. `+0x04 / +0x08` then `+0x0C / +0x10 / +0x14` then `+0x18`: that is position, rotation, FOV. The single most common camera in existence is the one in §15.1, and its fingerprint dominates the whole corpus.

The camera is in RCX or RDX half the time. Base-register distribution: **RCX 26.2% · RDX 26.1%** · RAX 7.9% · RBX 6.4% · RSI/ESI 7.5% · EAX 6.1%. RCX+RDX = **52%**; the top four = **66%**. That is not coincidence — it is the **Win64 fastcall convention**: RCX is `this` (a camera member function) or the copy **destination**, RDX is arg2 or the copy **source**. **When your cave captures a register, try RCX first, then RDX.** In the engine-sibling cluster (§15.1) it is exactly this: `rcx = dest POV, rdx = src POV`.

15.10 Engine identification from the module name

Across **261 distinct target modules**, the naming convention alone classifies a meaningful share of games before you open anything:

Module pattern	Count	Meaning
`*-Win64-Shipping.exe` `*-WinGDK-Shipping.exe`	**58** 22%	**The engine-sibling cluster.** You already have the AOBs (§15.1). **Check this first, always** — it is the single highest-value question you can ask about a new game.
`UnityPlayer.dll`	1 module 15 tables	Managed-runtime cluster (§15.2). **The signature is in the runtime DLL, so it is shared by every game on that runtime** — one module, many games.
`ShippingPC-*.exe`	1	Previous-generation UE. Int rotations (65536 = 360°), POV memcpy — family 6D.
Other engine DLLs (`Engine.dll`, `FC_m64.dll`, …)	22	**Hook the DLL, not the exe.** Signature is engine-wide → likely shared across that publisher's titles.
Bespoke `{game}.exe`	178	In-house engine. Expect full discovery — but check the constant headers (§15.6) first.

The asymmetry worth internalising. 22% of games are solved by *reading the exe's filename*. The remaining 68% of bespoke exes cost days each. **Spend the ten seconds on the filename check before anything else.**

15.11 FOV encoding — the decision table

Every mod hits this and it is easy to lose an hour on. The corpus's applied-FOV ranges (116 rows) settle it empirically: **FOV is stored in degrees far more often than anything else**, and the value itself tells you which.

Live value reads…	Encoding	Apply as
~30 – 160	**DEGREES** (dominant — nearly all applied rows)	`fov' = fov * Scale + OffsetDegrees`. Common observed windows: `[60,150]`, `[65,140]`, `[30,140]`, `[59,150]`. Typical native default **75**. Note these are usually **horizontal**.
~0.5 – 2.8	**RADIANS**	Convert once at the boundary; keep all mod-side math in degrees. (Rotation blocks confirm the tell: pitch bounded `±1.570796` = π/2, yaw `0 → 6.28318530718` = 2π.)
~0.3 – 1.7, and changing with zoom	**tan(FOV/2)**	`fov' = 2·atan(tan(fov/2) · k)`. **Do not scale it linearly** — that is a classic wrong turn. Distinguish from radians by checking whether `2·atan(v)` lands in a sane degree range.
~0.5 – 2.0, and *not* an angle	**FACTOR** (multiplier on a base FOV)	Multiply directly; find the base elsewhere (often a nearby constant or `DEFAULT_FOV_DEGREES`).
Integer, e.g. **128** constant	**FIXED/BYTE** (rare)	Sentinel/packed. Usually not the real FOV — look again.
Reads **0.0** or never changes	**NOT THE FOV FIELD**	Ship `NoFov=1` / `[FOV] Enabled=0` and identify the camera another way (orthonormal-pair test). **Precedent exists in the library** — one title's FOV field reads zero and the mod correctly ships with FOV disabled.

There is often more than one FOV. The corpus stores **separate world and player/weapon FOV expressions** for every game. If changing FOV moves the world but not the weapon (or vice versa), you have found one of two fields, not a bug. Also: some titles keep a **second FOV target field** the engine eases toward — write the live one and the target, or the engine will pull your value back.

15.12 The FOV may be a PAIR — raw plus a cached derivative NEW

§15.11 treats the encodings as alternatives: a field is degrees *or* radians *or* tan-half. A newly decoded engine shows they can be **both, in adjacent fields**, and that writing one without the other silently does nothing.

Reading the engine's own setter answers it outright — always disassemble the FOV setter before designing the write:

```
SetFov(this = rcx, fov = xmm1):
    movss [rcx+0xAC], xmm1        ; the FOV itself, in RADIANS
    mulss xmm1, [rip+...]         ; constant reads 0.5
    call  <tanf>                  ; callee loads 0x3fd45f306dc9c883 = 2/pi
                                  ;   -> the classic argument-reduction constant for TANGENT
    movss [rbx+0xB0], xmm0        ; tan(FOV/2)  <- a CACHE the renderer reads
```

Field	Meaning
`+0xAC`	FOV, radians. The value the game logic sets.
`+0xB0`	**tan(FOV/2)**, precomputed. What the projection actually consumes.

Write the raw field alone and your FOV change may do nothing at all — the renderer keeps using the stale cache. The tell is maddening: the struct visibly holds your value and the screen ignores it, which reads exactly like failure mode #1 (editing an upstream copy, §25.0.1) but isn't. **Always write the pair.**

The clue was in plain sight. The reference freecam for this engine moves a **QWORD** at `+0xAC` — 8 bytes, i.e. both floats — where a FOV is obviously 4. That oddity *is* the finding; a QWORD move on a float field means there is a second field next door that must travel with it.

The pair is also the best validator in this document. Checking `+0xB0 ≈ tan(+0xAC / 2)` is a **self-consistency test between two fields**, not a range check. Random data satisfies "is between 0.05 and 3.2" constantly; it essentially never satisfies a transcendental relationship between neighbours. When a struct offers a derived field, prefer it over any range fingerprint (§17.4).

16. The three camera struct layouts KEY FINDING

Corrected in Rev 4. The constant-header tree is **duplicated roughly fourfold** — 200 files, but only **49 unique games**, of which **33 declare both position and orientation**. Rev 3 quoted the raw file counts (55/28/26), which were inflated. The deduped counts are below. **The finding survives intact — the rank order and the three-way split are unchanged** — but the magnitudes are smaller and the honest sample is 33 games, not 200.

Raw offsets look chaotic (COORDS at `0x0`, `0xC0`, `0x90`, `-0xD8`, `0x574`, `0x41c`…). But the **delta between orientation and position** collapses almost everything into three layouts:

Δ	Games	Layout	Meaning & handling
+0xC	**10** 30%	`pos[3 floats]` then `rot` immediately	**Tight POV layout.** Position is 3 packed floats (12 B), rotation follows at once. This is the dominant modern-engine POV struct (§15.1): loc `+0x00/04/08`, rot `+0x0C/10/14`, fov `+0x18`. **Rotation is Euler, not a quaternion.**
+0x10	**8** 24%	`pos[vec4]` then `rot`	**SIMD-padded layout.** Position padded to 16 B (w often 1.0 — a free validator), then quaternion/rotation. Typical of engines using `movups` / `movaps` vec4 moves.
−0x10	**6** 18%	`rot[vec4]` then `pos`	**Rotation-first layout.** Quaternion at the anchor, position 16 B later. **This is exactly the vtable-returned render transform of the hardest title in the atlas** (quat `+0x00`, pos `+0x10/14/18`) — not a special case at all, but one of the three standard layouts.
−0x30	**4**	`rot` then `pos`, 48 B apart	A wider rotation-first variant — a 3×4 or padded rotation block ahead of the position. Treat as a −0x10 sibling.
+0x40	1	separated blocks	Position and orientation in different sub-blocks.
−0x24, +0x20, +0x470	1–2 ea.	outliers	Photomode/secondary structs, or genuinely unusual packing.

What to do with this. When you dump an unknown camera struct, you are not searching a huge space — you are **distinguishing three cases** — they cover **24 of the 33 deduped games (73%)**. Find one field (usually position: three plausible world coordinates that change as you walk), then test **+0xC, +0x10, −0x10** for the orientation. One of them is almost always it.

- Orientation at **+0xC** that reads like `±180` / `0..360` → **Euler degrees** (§7C).
- Orientation at **+0x10** or **−0x10** with 4 components of unit length in [−1,1] → **quaternion** (§7A).
- Orientation spanning 3 rows of ~unit-length vectors → **matrix** (§7B); check stride **0x0C vs 0x10**.
- Two world-space *points* a constant distance apart → **look-at rig** (§7D); expect no roll.

FOV offset clustering

`0x8` (18), `0x18` (10), `0x13C` (10), `0x7C` (9), `0x30` (9), `0x24` (9), `-0x78` (9), `0x6C` (8), `0xB0` (8). **FOV usually sits immediately after the transform block** — if you have found position and rotation, scan the next 8–16 bytes for a float in a plausible FOV range (see §17 for the range test).

17. The block/axis/validator schema KEY FINDING

The scan-profile corpus (77 games × 941 rows) encodes camera location as a **pure memory scan with no AOB at all**. This is a genuinely different locator family from everything in §9, and its central idea is directly reusable.

17.1 Structure

Each game defines up to **three blocks** — `Rotation`, `Position`, `FOV` — located independently. Each block has **four components**: `AxisX`, `AxisY`, `AxisZ`, `Extra` (the 4th/w/pad slot), each with a `ByteOffset` (relative to the block anchor, **may be negative**), an `ApplyType`, an `ApplyMultiplier`, and a `[MinValue, MaxValue]` range.

The `ScanType` code's leading digit is the block class:

ScanType	Block	Rows	Sub-code meaning (empirical, from unit & range correlation)
1xxx	Rotation	320	`1100` / `1101` = float Euler (radians or degrees) · `1050` / `1300` = **integer/fixed-point angles** (ranges ±32767, ±16383) · `1103` = components in [−1,1] → quaternion/basis vector
2xxx	Position	315	`2100` = float world units (ranges ±1.3e6, ±1e6, ±512000) · `2101` = smaller/fixed ranges (±16384, ±100000)
3xxx	FOV	306	`3101` dominant (294) · `3000` = byte/fixed encoding (constant 128 sentinels)

17.2 The validator constants — the real insight

A camera is identified by a RANGE FINGERPRINT across its components, not by an AOB.

The `[Min, Max]` pairs are not clamps. They are **the search predicate**: scan memory for a struct where every component simultaneously falls in its declared range. The tightest ranges do the identifying — and they sit on the components that are **never applied** (`ApplyType = 0`), which exist purely as validators.

```
Position  Extra  [0.9999 , 1.0001]     <- homogeneous w == 1.0        (10 games)
Position  Extra  [-0.0001, 0.0001]     <- padding w == 0.0            (23 games)
Rotation  Extra  [-0.001 , 0.001]      <- rotation vec4 pad == 0      (41 games)
Rotation  AxisZ  [-0.001 , 0.001]      <- ROLL IS ZERO                (56 games!)
FOV       Extra  [1.77777, 1.77779]    <- aspect ratio 16:9
FOV       Extra  [1.33332, 1.33334]    <- aspect ratio 4:3
FOV       Extra  [0.999  , 1.001]      <- a unit scalar
Rotation  AxisX  [-1.570796, 1.570796] <- PITCH bounded to +/- pi/2
Rotation  AxisY  [0 , 6.28318530718]   <- YAW spans a full circle
```

Read those last two lines carefully. They are how the block self-identifies which axis is which: **pitch is the one bounded to ±90°, yaw is the one that wraps a full circle** (`MinMaxWrap=true`), and **roll is the one pinned near zero** in 56 of 77 games because most games keep the horizon level. That is a free, engine-independent axis identifier.

Direct application to our mods. This validates and generalises our existing plausibility gates — `Ortho()`, `QuatPlausible()`, `IsUnitQuat()`, the look-at `w≈0 && dist==|eye−target|` gate, and the behavioural finder in the flagship mod. They are all instances of one idea. **Adopt the corpus's discipline: declare a range for every component including the ones you don't write, and reject the struct if any fails.** It is the cheapest dummy-camera rejection available and it costs nothing at runtime.

17.3 Scan-region and result discipline

The scan profiles also encode *where* and *how* to search — worth copying for any behavioural finder:

```
ScanPrivateMem / ScanExecutableMem / ScanImageMem / ScanHeapMem / ScanStackMem   ; region classes
MinRegionSize / MaxRegionSize           ; e.g. MaxRegionSize 100000  -> skip huge arenas
MinRegionStartFromExe / MaxRegionStartFromExe   ; bound the search RELATIVE TO THE MODULE
ScanAlignmentBytes                      ; usually 4 or 16
MaxValidResultCount / ...Hard           ; e.g. 20 candidates, hard-limit 1
ScanResultMajorityVote (+ Epsilon)      ; take the consensus candidate, not the first
PostScanOffsetType / Range              ; refine to a nearby anchor after the hit
CompareMaxDelta / CompareZType / CompareExtraType   ; frame-to-frame differential tests
ApplyMinDrawCalls                       ; e.g. 150 -> only apply once the frame is really rendering
ApplyGameYawCorrectionOffsetDeg         ; e.g. 90 -> engine's yaw zero is rotated vs ours
```

The most-common region set is `"Private, Executable"`. **`ScanResultMajorityVote`** is the notable idea: when several candidates match the fingerprint, take the **consensus** rather than the first hit — the same problem our per-pointer slot table (§8E) solves at apply time, solved here at locate time.

17.4 Identify by DATA when the code cannot be trusted NEW

§17 treats the validator fingerprints as a **sanity check** applied after you have found a camera. On a protected title they become something more useful: **the discovery mechanism itself.**

The situation that forces it: a modern anti-tamper exe can be **87% INT3 at rest**, with the real code decrypted into memory only at runtime. Then **every static technique in Part III is unavailable** — you cannot read a signature out of the file, cannot verify an inherited RVA, cannot even confirm the instruction exists. And a short pattern scanned at runtime is no help either: a 5-byte stack-spill signature hit **93 times** in one such exe.

THE PROBE: HOOK EVERYTHING, KEEP WHAT BEHAVES LIKE A CAMERA

```
1. Scan for the WIDENED instruction pattern (wildcard the displacement -
   the compiler may have moved the stack slot).          e.g. 0F 29 64 24 ??
2. Build a READ-ONLY cave on EVERY candidate. Spill the registers, bump a
   hit counter, restore everything, run the stolen bytes, jump back.
3. Watch. Score each candidate against the §17 fingerprint on LIVE DATA:
      |up| == 1          (a real unit vector)
      eye != focus       (a real look-at rig, sane distance)
      fov field plausible for its encoding (radians 0.05..3.2)
      a derived field agrees with its source (§15.12)  <- strongest
4. First candidate to pass N frames (~60) wins. Dispatch it to the real
   routine; unpatch the rest. NO re-patching on the hot path.
5. Report the winning RVA so the user can PIN it for instant startup.
```

Why this is version-proof in a way a signature is not. The fingerprint is a property of the *camera*, not of the build: an up vector is unit-length in every version that ever ships. Addresses move, bytes get re-encrypted, the compiler re-allocates the stack — none of it touches "the registers here behave like a look-at rig".

Safety property	Why it holds
Probes are **read-only**	They capture and score; they never edit the camera. A false candidate costs nothing but a counter.
Every candidate is **the same stolen bytes**	The scan matched one exact instruction form, so the steal length is right by construction at every site.
Refuse to guess	If no candidate proves the FOV field, print the scored table and abort. The geometry test alone (unit vector + two distinct points) is **weak** — lots of non-camera code passes it. Never accept it as the sole evidence.
Winner **dispatches**, others unpatch	Flip a flag and route the winning index to the real routine. Re-patching live code to "promote" the winner is an unhook race for nothing.

Test it before you need it. Force the probe path on a build you know works — set the RVA to 0, or corrupt one byte of the signature. That is an exact simulation of "the game just got patched". **A fallback you have never executed is not a fallback**; it is an untested claim, and the day it matters is the worst day to find out.

18. GPU / constant-buffer data (family 6E targeting)

When struct writes don't reach the screen, the mod must inject at the GPU boundary (§6E). The corpus supplies the targeting data directly.

18.1 Where the matrix lives

Field	Distribution across 209 entries
MatrixSizeBytes	**64** in 165 of 166 populated rows — i.e. a **4×4 float matrix**, essentially always.
BufferSlot	**0** (92) · 4 (29) · 1 (28) · 2 (26) · 12 (17) · 3 (8) · 6, 11 (rare). **Slot 0 is the first thing to try.**
MatrixByteOffset	**0** (76) · 64 (16) · 128 (11) · 224 (8) · 320 (7) · 1328 (6) · 160, 256 (4 ea.). **Offset 0 dominates**; the rest are multiples of 16/64 — i.e. matrix-slot indices.
BufferSizeBytes	4096 (44) · 3248 (21) · 608 (11) · 960 (9) · 2048 (8) · 1024 (8) · 512, 752, 912. **Buffer size is a strong identifier** — the corpus explicitly enables "search target buffer by size and by slot".
Inverse	33 of 209 need the matrix inverted before use.
HasCameraMemory	78 of 209 **also** have a CPU camera. 131 are **GPU-only** — for those, family 6E is the *only* route.

18.2 Shader constant names — what to look for

From 395 matrix locators: the overwhelmingly dominant constant name is `proj` / `Proj` / `PROJ` (241 of 395 — case varies). Then the combined matrices: `ViewProj`, `ViewProjectionMatrix`, `WorldViewProj`, `mvp`, `m_WVP`, `gWorldViewProj`, `ModelViewProj`, `c_mObjToClip`, `c_mAbsToClip`, `ToScreen`, `g_viewProjMatrix`, `m_P`.

API split: D3D9 312 / D3D11 83 in the locator table; the GPU profiles cover D3D9 (171), D3D11 (151), D3D12 (20), OpenGL (20). **CPU scan present in only 109 of 395** — again, most GPU-profile games have no usable CPU camera.

18.3 Matrix handling flags

```
MatrixTranspose      False 243 / True 88     ; ~27% need transposing
MatrixValidation     1 (172) / 0 (33) / 2 (2) ; validation on by default
MatrixTranslator     1 (266)                 ; translation-recovery mode
AllowMatrixRoll      present on 270 profiles  ; ROLL IS A PER-GAME PERMISSION, not a given
ReProjectMode        1 typical ; ReProjectInvertY
TransformUpdateMapType   4                   ; == D3D11_MAP_WRITE_DISCARD  -> the Map type to hook
CheckConstantBuffersOnlyOnce  true           ; cache the CB match; don't re-scan every frame
ConstantBuffersImmutableMono  true           ; immutable CBs are never the camera -> skip
UseIntermediateBuffer         true           ; stage edits, don't write the mapped ptr directly
PositionalTrackingMultiplier / ...ZMultiplier ; per-game lean scale (see §19)
```

Three things to steal for a 6E build. (1) `TransformUpdateMapType = 4` confirms the hook point: `Map` with `WRITE_DISCARD` is the CB upload path — our reference 6E mod hooks exactly `Map` (14) / `Unmap` (15) / `UpdateSubresource` (48). (2) **Cache the CB match** — identify the buffer once, then trust slot+size. (3) **Skip immutable CBs** — they cannot be the camera. Together these turn a per-frame brute-force scan into a near-free lookup.

19. Aggregate tuning defaults (370 games)

Empirical priors for a new mod's INI, from the head-tracking parameter table.

Parameter	Value	Reading
Look sensitivity	median **0.86** mode **1.0** (152)	1:1 is the most common choice; the median dips below 1 because third-person games need damping. **Our library's mods cluster at 0.6–0.7** — a reasonable, slightly conservative default.
Vertical multiplier	**1.0** (62/67)	Pitch is almost never scaled differently from yaw. Don't over-engineer.
Positional tracking	enabled **315/370**	**85% of games support positional lean.** Enable by default.
Pos-track user multiplier	**1.0** (342/343)	The *user* multiplier is a trim; the real scale is per-game (below).
Per-game lean scale	varies wildly	From the GPU profiles' `PositionalTrackingMultiplier`: e.g. **5.18** for one open-world title, **2.8** for another, **7.0** elsewhere; Z-multiplier commonly **0.3**. **This is the "world units per metre" constant we tune by hand** — the corpus has it precomputed for 342 games.
Roll enable	True **134/134** populated	Where roll is declared at all, it's allowed — but note it's declared for only 36% of games, and the GPU profiles gate it separately (`AllowMatrixRoll`). **Consistent with §7D: roll is not universally available.**
InvertX / InvertY	False **365/370**	Inversion is the exception, not the rule — but our mods invert far more often, because we *add* to a game camera rather than replace it. Expect to flip signs.
Crouch / jump thresholds	**−0.1 m** / **+0.05 m** (338/338)	Universal constants for gesture detection from head Y. **Not currently used by any of our mods** — a possible future feature (duck your head to crouch).
Smoothing	0 (only 3 rows)	Essentially unset across the corpus — smoothing is left to the tracker. **Our spring filter (§12) is an improvement over the corpus baseline**, not a copy of it.
Yaw correction	e.g. **90°**	`ApplyGameYawCorrectionOffsetDeg` — some engines' yaw zero is rotated relative to the tracker's. If yaw is right but rotated by a quarter turn, this is why.

FOV expressions

The corpus stores FOV as an **expression**, not a number — because vertical/horizontal conversion depends on aspect:

```
DEVICE_VERT_TO_HOR(4/3)        ; convert the device's vertical FOV to horizontal at 4:3
DEVICE_VERT_TO_HOR(16/9)
DEVICE_VERT_TO_HOR(1.777778-((1.777778-%BASPECT%)*0.5))   ; blend toward the game's own aspect
%DFOVV%                        ; the device vertical FOV verbatim
```

Separate `FovWorldExpr` and `FovPlayerExpr` exist because the **world and the player/weapon are often rendered with different FOVs**. Relevant to us mainly as a warning: if a game's FOV field seems to have no effect on some geometry, there is probably a second FOV.

20. Corpus lookup workflow

BEFORE WRITING ANY CODE FOR A NEW GAME

```
1. IS IT AN ENGINE SIBLING?
   Does the exe end in -Win64-Shipping / -WinGDK-Shipping, or does it load a
   known managed runtime DLL?
       YES -> you already have the AOBs (§15.1 / §15.2). Skip to step 6.

2. IS THERE A CONSTANT HEADER?  (200 games)
   -> struct offsets, intercept RVAs, steal lengths, native FOV, in plain text.
      Note: offsets are often NEGATIVE from the anchor.

3. IS THERE A SCAN TABLE?  (596 tables, 882 unique AOBs)
   -> aobscanmodule(...)  = the AOB + module
      assert(...) / [DISABLE] db = the FULL original bytes -> steal length + offsets
      code: block                = the capture register and the GATE
      trailing comment           = injection-point RVA + disassembly

4. IS THERE A TOOL BINARY?  (19+)
   -> strings for AOB * labels; pattern sits before the label in .rdata;
      find the stub by its re-embedded stolen bytes; disassemble for offsets.

5. IS THERE ONLY A GPU PROFILE?  (131 of 209 are GPU-only)
   -> no CPU camera exists. Go straight to family 6E.
      CB slot (try 0), buffer size, matrix byte offset (try 0), transpose, inverse.

6. CROSS-CHECK THE PRIORS
   -> lean scale (PositionalTrackingMultiplier), native FOV, yaw correction,
      roll permission, aspect/FOV expression.

7. PICK: hook family (§6) x camera math (§7) x drive model (§8).
   Then build wobble-first (§10).
```

Coverage reality check

If the corpus gives you…	Then…
A shared engine-family AOB	**Best case.** Camera solved before you start. Adapt the gate from "skip" to "add" and go.
A constant header	**Very strong.** Offsets + steal lengths in text. Verify the AOB still matches your build.
A per-game table only	**Strong.** Decode the `[DISABLE]` bytes for the struct; re-derive the AOB against your exe (table AOBs go stale across patches — ours must be wildcarded).
A scan profile only	**Usable.** You get the block layout, offsets and the validator fingerprint — but no code site. Build a behavioural finder (§9.5) using the ranges as the predicate.
A GPU profile only	**Family 6E is the route.** Do not waste time hunting a CPU camera that may not exist.
Nothing	Full discovery: find-what-writes (§13), reference-binary RE (§15.5), or a behavioural finder.

Three standing cautions when using corpus data.

- **Table AOBs are usually not wildcarded** — they were written for one build. Ours must be. Wildcard the drifting operand (the `?? 1?` idiom, §15.1), keep opcodes exact.
- **A freecam's camera is not always the render camera.** A tool that "works" may drive an upstream object that the engine happens to consume — that guarantee doesn't transfer to an *additive* mod. Wobble-test regardless.
- **Freecam offsets can be photomode/debug structs.** Check for a parallel struct (e.g. a photomode block at a large offset) before assuming you have the gameplay camera.

PART IV — THE STANDARD

21. Standard config — the reference INI

The flagship open-world mod's INI is the project standard. Every new mod adopts this structure, these names, these defaults, and this documentation style. It is reproduced here in full as the canonical template.

> **Read §21.4 before adopting.** This INI is the best-*designed* config in the library, and its **structure** is the standard. Its **key names**, however, are a minority dialect — measured across all 51 shipped INIs, `[OpenTrack] Port` is used by 5/51 and `[Network] UDPPort` by 28/51. Adopt the structure and the spelling for *new* mods, and add the alias fallback (§21.5) so the other 46 keep working.

> **Why this one is the structural standard.** It is the only INI in the library that gets all five of these right: **(1) progressive disclosure** — a Quick Start at the top, then FEEL/TUNING, then a clearly fenced ADVANCED section the user is told not to touch; **(2) every key has a plain-English comment** saying what it does and which way to move it; **(3) a Mode selector** with a documented fallback ladder; **(4) the self-test is fenced off** and labelled "leave OFF for real tracking"; **(5) it states the defaults are already correct** — "Nothing else needs changing." That last line matters more than any single key.

21.1 The canonical template

```
; ============================================================================
;  {GAME} - 6DOF Head-Tracking   (OpenTrack -> real render camera)
;  Full 6 degrees of freedom in gameplay AND cutscenes.
;
;  QUICK START
;    1. In OpenTrack set Output = "UDP over network", IP 127.0.0.1, port 4242.
;    2. Launch the game and play. Tracking starts automatically once the
;       camera is located (a few seconds after the world loads).
;    3. Hotkeys:  END = pause/resume tracking     HOME = recenter
;
;  Nothing else needs changing. The sections below are for tuning the feel.
; ============================================================================

[Tracking]
; 10 = render-camera hook (default): full 6DOF everywhere, incl. cutscenes.
;  5 = native/script camera (gameplay only, no memory writes).
;  0 = diagnostics only (locate the camera, write nothing).
Mode=10
; Master on/off. 1 = tracking on at launch (END toggles it in-game).
Enable=1

[OpenTrack]
; UDP port OpenTrack sends to. Must match OpenTrack's Output port.
Port=4242

; ----------------------------------------------------------------------------
;  FEEL / TUNING
; ----------------------------------------------------------------------------

[Rotation]
; How far the view turns per unit of head turn. 1.0 = 1:1. Raise for more.
YawSensitivity=0.7
PitchSensitivity=0.7
RollSensitivity=0.7
; Flip an axis if it turns the wrong way (0 = normal, 1 = inverted).
InvertYaw=1
InvertPitch=0
InvertRoll=1
; Hard cap on head rotation (degrees).
LimitDeg=35

[Position]
; 6DOF lean - moving your head shifts the camera in world space.
Enabled=1
; Overall lean strength (metres of camera travel per metre of head motion).
LeanScale=5.0
; Max lean distance (metres) so you can't clip through walls/the player.
LeanLimit=3.5
; Per-axis fine tuning (-1 = use LeanScale). X = side, Y = up/down, Z = in/out.
LeanScaleX=-1
LeanScaleY=-1
LeanScaleZ=-1
InvertX=0
InvertY=1
InvertZ=1
; Swap head up-down (Y) with in-out (Z). 1 = head up/down moves camera up/down (recommended).
SwapYZ=1

[FOV]
; Optional field-of-view control applied to the render camera. Off by default.
Enabled=0
; Multiply the game's FOV (1.0 = unchanged, 1.1 = ~10% wider).
Scale=1.0
; Add a fixed amount of FOV, in degrees (can be negative).
OffsetDegrees=0.0
; Dynamic zoom: lean toward the screen to narrow FOV (degrees per metre of
; forward lean). 0 = off. Try 15-30 for a "lean in to zoom" effect.
LeanZoom=0.0
; Clamp so FOV can never go out of range.
Min=20.0
Max=120.0

[Smoothing]
; 1 = critically-damped spring (recommended), 0 = simple EMA.
SmoothMode=1
; Spring stiffness in Hz. Higher = snappier/less lag, lower = smoother.
SmoothHz=8.0
; EMA factor (only used when SmoothMode=0). 0..1, higher = snappier.
Smoothing=0.6

[Hotkeys]
Enabled=1
; Virtual-key codes. END=0x23, HOME=0x24.
ToggleKey=0x23
RecenterKey=0x24

; ----------------------------------------------------------------------------
;  SELF-TEST (leave OFF for real tracking)
; ----------------------------------------------------------------------------
[Test]
; 1 = the view moves by itself (no tracker needed) so you can confirm the mod
;     works. Set 0 for real head-tracking. OFF by default.
```

```
TestWobble=0
WobbleMode=1
WobbleDeg=12
WobbleLean=0.25
WobbleHz=0.35
WobbleAxis=0

; ------------------------------------------------------------------------
;  ADVANCED - you normally never touch these
; ------------------------------------------------------------------------
[Hunter]
; Locates the on-screen render camera by float-differential each session and
; keeps it locked (with a fast cached-cluster re-lock if it ever switches).
Background=1
ChainScan=1
; Diagnostic writer-finder (page write-protect + single-step). One-time tool.
; Leave 0 for normal play.
AutoWriter=0
Backtrace=0
WriteTest=0

[General]
; Write HeadTracking.log next to the game exe.
Diagnostics=1

[Hook]
; Track every on-screen camera - loading screens, scripted cams, gameplay AND
; cutscenes (1=on, 0=only the located render cam).
AllCameras=1
; If the mod tracks a secondary camera / feels wrong on some cameras, set
; OnScreenOnly=1: edits ONLY the camera the game is actually rendering
; (falls back to all during loading).
OnScreenOnly=0
```

21.2 Sections a per-game mod adds

The template above is the **universal** part. Depending on the camera math and locator, add:

```
[Matrix]                        ; matrix-convention games (§7B)
MatrixOffset=0x0                ; orientation matrix inside the captured object
Transpose=0                     ; rows vs columns basis
ViewMatrix=0                    ; 0=world, 1=view (negates rotation AND lean)
PosRow=3                        ; matrix row holding translation (row3 = +0x30)
RotStride=16                    ; 16 = vec4 rows; 12 = tight 3x3 (see §13)
MatrixMode=0                    ; 0 = left-multiply (column-vector view), 1 = right

[Rotation]                      ; add for per-axis mapping (§7A/§7B)
YawAxis=2  PitchAxis=0  RollAxis=1   ; head angle -> basis row (0=right 1=up 2=fwd)

[Position]                      ; add where the engine's unit scale matters (§7D)
WorldUnitsPerMetre=1.0          ; game units per metre of head movement
LeanDeadzone=0.02               ; metres; kills resting bias + micro-shake

[FOV]                           ; add where encoding is non-obvious (§11)
FovOffset=0x60                  ; fov field inside the captured object
Encoding=0                      ; 0=degrees 1=radians 2=tan_half 3=factor
Degrees=0                       ; >0 pins an absolute vertical FOV (gameplay)
DegreesCutscene=0               ; 0 = KEEP the cutscene's own (lower) FOV
CutsceneDropDeg=8               ; native FOV below (baseline - this) => cutscene
CutsceneDebounceMs=250
CutsceneTrackScale=1.0

[Hook]                          ; the version-agnostic locator (§9)
UseAOB=1
AOB=F2 0F 11 01 48 8B DA 8B 42 08 89 ??     ; ?? = wildcard
AOBBpOffset=0
Steal=10
OffPos=0x00  OffRot=0x0C  OffFov=0x18
CamCopy=0x0                     ; static fallback (used ONLY if UseAOB=0)

[Advanced]                      ; validity gates (§17)
ValidateLookat=1
WarmupFrames=8
MaxOrbitDistance=0              ; 0 = off; skip far aux/reflection cameras
DumpStruct=0                    ; log the struct to re-confirm offsets live
```

21.3 The levers, by what the user is trying to fix

"It..."	Lever
doesn't do anything	`[Test] TestWobble=1` first. If wobble works, the hook is fine and it's OpenTrack. If not, §13.
turns the wrong way	`InvertYaw` / `InvertPitch` / `InvertRoll`
turns too much / too little	`YawSensitivity` / `PitchSensitivity` / `RollSensitivity`
turns diagonally / skews	`YawAxis` / `PitchAxis` / `RollAxis` permutation, or `Transpose`, or split-axis (§7B)
lets me turn too far	`LimitDeg`
leans the wrong way	`InvertX` / `InvertY` / `InvertZ`, or `SwapYZ`
barely leans / leans miles	`LeanScale`, then `WorldUnitsPerMetre`
clips through walls	`LeanLimit`
shakes / jitters	`SmoothHz` down (8 → 5), `LeanDeadzone` up. If it oscillates in/out, it's the engine fighting you (§7D) → `WorldUnitsPerMetre` down.
feels laggy	`SmoothHz` up (8 → 12)
drifts / sits off-centre	`HOME` while seated naturally. Persistent → `LeanDeadzone`.
is the wrong FOV	`[FOV] Enabled=1`, then `Scale` or `OffsetDegrees`
ruins cutscenes	`DegreesCutscene=0`, `CutsceneTrackScale` down
tracks the wrong camera	`OnScreenOnly=1`, or `AllCameras=0`, or `MaxOrbitDistance`
broke after a patch	`[Hook] AOB=` — re-derive and paste. **No rebuild.** This is the whole point of version-agnosticism.

21.4 The config surface has drifted — and the standard is a minority dialect MEASURED

Auditing all **51 shipped INIs** key-by-key produced the most actionable finding of this revision: **there is no shared config vocabulary.** The same concept has up to four names, and the INI promoted above as the standard is, by key names, **not the majority dialect**.

Concept	Spellings in the wild	Verdict
UDP port	`[Network] UDPPort` — **28/51** `[OpenTrack] Port` — 5/51 (the "standard") `[Net] …` — 5/51 other port key — 9/51	**4 spellings.** The most basic setting in the entire project — the port the tracker sends to — is spelled four ways. The standard's spelling is used by **10%** of mods.
Rotation gain	`YawMultiplier` — **32/51** `YawSensitivity` — 17/51 (the "standard") other yaw-gain key — 23/51	`Multiplier` **is the incumbent 2:1.**
Lean gain	`LeanScaleX/Y/Z` — 21/51 `LeanScale` — 17/51 `WorldUnitsPerMetre` — 17/51 `SensitivityX/Y/Z` — 15/51	**Four overlapping keys** for one idea, and some mods ship several at once.
Lean clamp	`LeanLimit` — 20/51 `LimitX/Y/Z` — 14/51	Scalar vs per-axis; both legitimate, neither universal.
Self-test	`TestWobble` — 8/51 `WobbleTest` — 6/51 `WobbleMode` — 9/51 · `WobbleAxis` — 9/51	**The flagship and the template disagree on the name of the flagship feature** — the two words are simply transposed.
Smoothing	`[Position] Smoothing` — 15/51 `[Filter]` — 7/51 `[Smoothing]` — 4/51	Smoothing is most often **buried inside** `[Position]` — where a user will never find it.
Locator	`[Hook]` — 7/51 · `[Camera]` — 6/51 `[CamState]` — 4/51 · `[Hunter]` — 2/51	Four names for the version-agnostic block.
FOV	**22 distinct keys**; the most common (`FovOffset`) is in only **7/51**	**The worst offender by far.** `FovScale` / `Scale` , `FovDegrees` / `Degrees` , `OffFov` / `FovOffset` , `IsRadians` / `Encoding` , `FovAdd` / `OffsetDegrees` … all coexist.

Why this matters more than it looks. Users move between our mods. A person who learns `YawMultiplier` on one game opens another and finds `YawSensitivity` ; the file looks familiar, the key silently does nothing, and the mod appears broken. **Every one of these is a support burden with no engineering benefit.** And it directly undercuts the version-agnostic promise: "just edit the INI" only works if the user can find the key.

21.5 The migration rule: alias, don't rename

Do not mass-rename 51 INIs. That breaks every config a user has already tuned, for no functional gain. Instead every mod's config reader adopts a **two-line alias fallback**: read the standard name, fall back to the legacy name, default last. New INIs ship the standard spelling; old INIs keep working untouched.

```
// read standard name; fall back to the legacy name; then the default.
static float cfg2(const char* sec,  const char* key,
                  const char* sec2, const char* key2, float def, const char* ini)
{
    char buf[64], dbuf[64];
    snprintf(dbuf, 64, "%g", def);
    // 1. the standard spelling
    GetPrivateProfileStringA(sec, key, "\x01", buf, 64, ini);
    if (buf[0] != '\x01') return (float)atof(buf);
    // 2. the legacy spelling
    GetPrivateProfileStringA(sec2, key2, dbuf, buf, 64, ini);
    return (float)atof(buf);
}

// usage — the whole migration, per key:
g.port      = (int)cfg2("OpenTrack","Port",   "Network","UDPPort",   4242, ini);
g.yawGain   =      cfg2("Rotation","YawSensitivity", "Rotation","YawMultiplier", 0.7f, ini);
g.testWob   = (int)cfg2("Test","TestWobble",  "Test","WobbleTest",      0, ini);
```

21.6 The canonical alias table

The mapping every mod adopts. **Left is what new INIs ship and what documentation refers to. Right is what must still be accepted**, forever, silently.

Standard (write this)	Legacy aliases (still accept)	Notes
`[OpenTrack] Port`	`[Network] UDPPort`, `[Net] Port`, `[Net] UDPPort`	Default 4242 either way.
`[Rotation] YawSensitivity` `PitchSensitivity` · `RollSensitivity`	`YawMultiplier` · `PitchMultiplier` · `RollMultiplier`	Identical semantics; pure rename.
`[Rotation] LimitDeg`	`LimitDegrees`	—
`[Position] LeanScale`	`SensitivityX/Y/Z` (scalar use)	Scalar master gain.
`[Position] LeanScaleX/Y/Z`	`SensitivityX/Y/Z` (per-axis use)	`-1` = inherit `LeanScale`.
`[Position] LeanLimit`	`LimitX/Y/Z`	Keep per-axis limits **as well** where a game needs them — they are not redundant, just differently scoped.
`[Position] WorldUnitsPerMetre`	— (keep)	**Not an alias of LeanScale.** Engine unit conversion; orthogonal to user gain. Both belong.
`[Smoothing] SmoothMode` `SmoothHz` · `Smoothing`	`[Filter] SmoothMode` / `SmoothHz` / `SmoothEMA`, `[Position] Smoothing`	**Promote out of `[Position]`.** Smoothing is not a position setting.
`[Test] TestWobble`	`WobbleTest`	Transposed words — pick one forever.
`[Test] WobbleMode` · `WobbleAxis` · `WobbleDeg` · `WobbleLean` · `WobbleHz`	`WobblePos` → `WobbleLean`	Already near-consistent.
`[FOV] Enabled` · `Scale` · `OffsetDegrees` · `Degrees` · `LeanZoom` · `Min` · `Max` · `Encoding`	`FovScale` · `FovAdd` / `FovOffset` · `FovDegrees` · `FovFromZ` · `FovClamp` · `IsRadians` · `NoFov` (inverse of `Enabled`)	**The big one — 22 keys collapse to 8.** Note `FovOffset` is dangerously overloaded: in some mods it is the *struct offset*, in others a *degrees offset*. **Standard: `[Hook] OffFov` = struct offset; `[FOV] OffsetDegrees` = degrees.**
`[Hook]`	`[Camera]`, `[CamState]`, `[Cave]`	The version-agnostic locator block.
`[Hunter]` / `[Finder]`	— (keep separate)	**Not an alias of `[Hook]`.** A behavioural finder is a different locator class (§9.5).
`[General] Diagnostics` · `AutoEnable`	— (already standard: 38/51 · 32/51)	**The two keys that are already near-universal.** Leave alone.

The one genuine collision to fix in code, not aliases: `FovOffset` means *struct byte offset* in some mods and *degrees to add* in others. An alias cannot disambiguate that — the same key name means two different things with two different units. It must be resolved by **renaming the struct-offset use to `[Hook] OffFov`** (already the spelling in 5 mods) and reserving `[FOV] OffsetDegrees` for the angle. This is the only rename in the table that is not purely cosmetic.

22. Standard feature set

The definition of "done". Every new mod ships all of these; every old mod is a candidate for backfill.

	Feature	Default	Requirement
1	**Version-agnostic locator**	`UseAOB=1`	AOB with wildcards on drifting operands → runtime base → relative offsets. Static address only behind `UseAOB=0`. Install **verifies bytes and aborts safely** on mismatch — a patch degrades to "no hook", never a crash.
2	**Full-6DOF wobble self-test**	`WobbleMode=1`	2.5 s dwell on each of yaw → pitch → roll → lean X → lean Y → lean Z, axis named in the log. Confirms the hook on the desktop with no tracker. **Ship it enabled during bring-up.**
3	**Full 6DOF**	on	Rotation (3) + lean (3), per-axis mapping, per-axis invert, per-axis scale, clamps.
4	**Spring smoothing**	`SmoothMode=1` `SmoothHz=8`	Critically-damped spring on **all six channels**. `dt` clamped [0.5 ms, 100 ms]; state seeded on first frame. EMA retained as legacy `SmoothMode=0`.
5	**Static FOV options**	`Enabled=0`	Scale, absolute degrees, offset, lean-zoom, min/max clamp, encoding-aware (deg/rad/tan-half/factor). Projection-consistent where we own the matrix.
6	**Cutscene keeps its own FOV**	`DegreesCutscene=0`	Detector (§11) with debounce; **self-disables** when the FOV field isn't plausible. Optional `CutsceneTrackScale`.
7	**Hotkeys + toggle**	`END` / `HOME`	Toggle and recenter, remappable by VK code, and **disableable** (`[Hotkeys] Enabled=0`) for users whose games bind those keys.
8	**Recenter from first real packet**	—	Never the all-zero default. No start-tilt.
9	**The gate + dummy rejection**	—	Only the render camera is edited. Range-fingerprint validators on every component (§17), including ones we don't write.
10	**Per-pointer slot table**	16 slots	Mandatory when several cameras share the hook.
11	**Warm-up + discontinuity guard**	8 frames	No writes during init/transition; stop writing ~90 frames on a scene cut and re-sync.
12	**Single-instance mutex**	—	Receiver starts only after the hook installs.
13	**Diagnostics log**	`Diagnostics=1`	Match address, applies/sec, recv state, head pose, live struct row — enough to debug from a user's log alone. **Every hard problem in this project was solved from a log.**
14	**Mode ladder**	best-first	Where a safer route exists (native/script camera, polling fallback), keep it as a lower Mode so a broken AOB degrades instead of dying.
15	**Standard config vocabulary**	alias fallback	Ships the standard spelling (§21.6) and **silently accepts every legacy alias** so no user's tuned INI ever breaks. Smoothing lives in `[Smoothing]`, never `[Position]`. `[Hook] OffFov` = struct offset; `[FOV] OffsetDegrees` = angle.
16	**Complete package**	—	`.asi` + proxy + `.ini` + `license.txt` + `README.md` + full `src/`. Rebuilds to the same byte size.

PART V — REFERENCE

23. Per-game atlas

51 packages audited. Status: ✅ working/perfect · ⚠ working with caveats · ⏸ on hold. Arch is the game exe's.

Assassin's Creed / AnvilNext

Game	Arch	Hook	Camera / offsets	St.
AC Origins ACOrigins.exe	x64	6A sync-cave Denuvo-safe	pos `+0x60` (mirror `+0x2E0`), quat `+0x70` (mirror `+0x2F0`), fov `+0x264`. `quatMode=1` (Z-up). Convention X=right, Y=forward, Z=up. **Hook the LAST camera write of the frame** (IGCS WRITE2): AOB `0F 58 E5 0F 29 26 41 0F 29 24 24 F3 41 0F 10 07`, steal 16, camera = `r12 − 0x70`. At WRITE2 both clean values are already in memory and nothing overwrites after → one cave suffices for full 6DOF. Photomode struct (coords +0x470, quat +0x480) left alone.	✅
AC Unity ACU.exe	x64	vtable hook + 6B fallback	chain: `manager → *(*(manager+0x40)) → transform = camObj+0x50`; pos `+0x00`, quat `+0x10`, fov `+0x20`. Active camera = the one whose quat changes most over two samples; `QuatPlausible()` gate.	✅
AC Unity — Smooth	x64	vtable hook	The choppiness fix (§6B): HW-BP/VEH demoted to fallback-only. **Prefer this build.**	✅
AC Syndicate ACS.exe	x64	chain (as Unity)	**Identical to Unity — one codebase**, per-game AOB. Ships separately.	✅

Crystal Dynamics / Foundation

Game	Arch	Hook	Camera / offsets	St.
Shadow of the Tomb Raider	x64	6A sync-cave steal 34 (0x22)	coords `+0x80`, quat `+0xA0`, fov `+0xB0`. AOB `66 0F 7F 81 80 00 00 00 0F 28 8A A0 00 00 00 0F 29 89 A0 00 00 00 8B 82`. Steal = 34 (same as IGCS), ends just before `mov eax,[rdx+0xB4]`. **Second capture-cave** at the address-select site (RDX = the one gameplay camera). Per-axis quat builder; recenter from first real packet. REV13 "intensity10".	✅
Rise of the Tomb Raider	x64	6A sync-cave	Same family: `+0x80` / `+0xA0` / `+0xB0`, steal 34.	✅
Tomb Raider 2013	x86	pointer-chain	matrix @ `cam+0x08` (VIEW). AOB `D9 5E 88 53 8B CF E8 ?? ?? ?? ?? D9 5E 90`. Carries `WobbleAxis` **and** `SmoothCD`.	✅

Frostbite / Eclipse / BioWare

Game	Arch	Hook	Camera / offsets	St.
Dragon Age: Origins DAOrigins.exe	x86	6B VEH HW exec-BP	eye `+0x10`, proj `+0x40`, VIEW `+0x80`, WORLD `+0xC0`, near/far `+0x130`, fov `+0x140`. **FOV field valid** (~0.3–2.5 rad) → projection-consistent FOV (`P[0]/P[5]` + `F` rescaled together), `FovFromZ`. EMA + yaw-only wobble. Static fallback `CamCopy=0x81E810` behind `UseAOB=0`. Shipped `.asi` **384,690** B.	✅
Dragon Age 2 DragonAge2.exe	x86	6C inline	Same offsets. **`+0x140` reads zero → ships `NoFov=1`**; picks the camera with a VIEW/WORLD orthonormal-pair test instead. 16-slot base table. Shipped `.asi` **378,285** B. (Source header + `moduleName` default still say "DAOrigins.exe" — cosmetic, §17.)	✅
Dragon Age: Inquisition	x64	write-hook	ROT `0x00` / POS `0x30` + **DUPLICATE** ROT2 `0x40` / POS2 `0x70`. **Write both** or the renderer reads the stale one — the gotcha of this title. Real cameras have nonzero pos at +0x30. Polling fallback shares the same math path.	✅

Northlight (Remedy)

Game	Arch	Hook	Camera / offsets	St.
Control (+ Steam variant)	x64	6C inline (AOB) into the renderer DLL	`quatOff` / `posOff` / `fovOff` all INI-configurable (default −1 = auto). **Cutscene by hook-staleness**: `renderMainView` idle >120 ms == cutscene (rmv freezes in cutscenes, `bom` does not); `fovHysteresisMs=350`. v4.2 `unifiedView` applies at `svtw` to the at-camera view every frame (gameplay + cutscene + conversation). `fovAddCutscene` **deprecated** — one FOV value everywhere. 1087 lines.	✅
Alan Wake 2 (+ V2)	x64	6A sync-cave SetViewAndFov	**The pitch lesson.** The earlier build hooked the IGCS gameplay-camera copy: yaw and roll stuck, **pitch did not** — the 3rd-person controller re-derives the forward component downstream (logs: `wroteFwdY 0.39 → gameLeftFwdY 0.000`). Independently confirmed by the AW2 VR mod, which gets full 6DOF only by hooking the engine's final view+FOV setter. This build hooks **SetViewAndFov**. Tight 3×3 stride `0x0C`. Both `.asi` 400,698 B (V2 repackage).	✅
Alan Wake Remastered Game_f_x64_EOS.exe	x64	6A sync-cave	rows `+0x00` / `+0x0C` / `+0x18` (**stride 0x0C**), eye `+0x24`. Hook: unique suffix `45 33 C9 48 8D 4B 08` @ RVA `0x15DFDE` (after the matrix-writer call), `rbx` = base. **Ships a `dbghelp.dll` proxy** — the exe imports neither dinput8 nor version. FOV is **tan-half**, base 90° vfov. `WorldUnitsPerMetre=60`. Yaw inverted, pitch not (933-sample controller correlation).	✅
Quantum Break	x64	6A sync-cave	**12 DOUBLES** (not floats) — camera→world matrix at `camStruct+0x70..0xC8`. AOB `4C 8B 05 ?? ?? ?? ?? 49 8D 50 20 8B 08 89 0A F3 0F 10 40 04 ... 48 8D 4D 90`; `cam_ptr = match+7+int32@(match+3)`; `instr_vm_write = match+0x90` (144-byte block); return = `+0x90`. Carries `SmoothCD`. Decoded from the exe + Heebo's CT.	✅

Dawn / Glacier (Eidos-Montréal)

Game	Arch	Hook	Camera / offsets	St.
Deus Ex: Mankind Divided DXMD.exe	x64	**Two hooks** write + FOV-read	Write hook snapshots the clean camera (game thread); FOV-read hook applies once per frame. **Idempotent** → safe at both sites. Key insight: `dt` is consumed once per frame, so the second hook sees `dt ≈ 0` and the smoothing state does **not** double-advance. 3×3 packed tightly (9 contiguous floats @ `+0x50`), coords @ `+0x74`. Z-up / Y-into-screen. FOV Scale/Degrees/Min. v2 has cutscene logic.	✅
Guardians of the Galaxy GOTG.exe	x64	6A sync-cave steal 38	4×4 row-major @ `camera+0x20`: row0 `+0x20`, row1 `+0x30`, row2 `+0x40`, pos `+0x50`, extra float `+0x60`. `rbx` = camera. **Single hook covers rotation AND position** (one contiguous copy). AOB extended past IGCS's 6-instruction locator through the +0x50 store and +0x60 float = 38 bytes, all `reg+disp`, fully relocatable. 14-byte abs jmp. **Same engine as DXMD** → defaults taken from DXMD: `YawMultiplier=-1`, `InvertY=1`, `WorldUnitsPerMetre=10`.	✅
Deus Ex: Human Revolution DXHRDC.exe	x86	**6E D3D11 CB** injection (V2)	**The GPU-injection reference.** Hooks `Map` (14)/ `Unmap` (15)/ `UpdateSubresource` (48); finds the view matrix inside the CB by matching rot AND translation against the live matrix from the chain `CameraManager = *(base+0x187CEC0) → *(mgr+0x30) → view @ +0x80`. Handles direct and transposed uploads. Game memory never touched → no race, no flicker. (V1 is the older build; **prefer V2**.)	✅

PlatinumGames / Level-5 / look-at rigs

Game	Arch	Hook	Camera / offsets	St.
NieR: Automata NieRAutomata.exe	x64	6A capture-cave + apply off-thread	**Look-at rig (§7D).** eye `+0xF0`, target `+0x100`, distance `+0x134` == |eye−target|, FOV `+0x138` (rad ≈ 50°), euler `+0x40/44/48` (**consumed upstream — writing is a no-op**), zoom `+0x130` (**do not write**), `+0x60` = **POINTER, do not write** (crashes inside FUNCTION A). Y-up. Capture AOB `F3 0F 11 8F 38 01 00 00 F3 0F 10 8F 38 01 00 00 48 8D 4F 60 E8`, steal 16. **Absolute-on-live** drive model. **No roll DOF.** Validity gate (w≈0 + distance match) + 8-frame warm-up + discontinuity guard (~90 frames) + scoped VEH/ `longjmp` guard. dual-field FOV (`0x138` / `0x188`). Tuned: sens −0.12, `WorldUnitsPerMetre=1.0`, `LeanDeadzone=0.02`, Smoothing 0.85. dinput8 proxy.	✅
NieR Replicant ver.1.22474487139	x64	6A sync-cave	**3×4 row-major affine** at base: rotation 3×3 in the first three floats of rows `+0x00/+0x10/+0x20`; **translation in the 4th column** (`+0x0C/+0x1C/+0x2C`). Engine cycles **three** camera matrices — index in `rcx`; **`rcx == 0` = main render camera**. Second matrix from `r9` at `[rcx+0x30..]` left alone. No FOV in the view matrix → no FOV wired. dinput8 proxy. (Two packages: `-v0` has the `.asi`, the other has the source — §17.)	✅
Ni no Kuni Remastered NinoKuni_WotWW_Remastered.exe	x64	6A sync-cave view-matrix (4th pass)	**The five-pass drive-model saga (§8B) — the most instructive log in the library.** Final: hook `0x5B5CF8` (the END of the view builder, where the game writes the final 4×4 to `[rdx]`), AOB `66 0F 7F 12 66 0F 7F 4A ?? 66`, steal **28** (`0x1C`), stolen-first, `rdx` preserved. **Source-confirmed** against open IGCS NNK1 (`MATRIX_IN_STRUCT_OFFSET=0x000`, `COORDS_IN_STRUCT_OFFSET=0x030`). Column-vector view matrix → head rotation applied LEFT (`MatrixMode 0`). **Split-axis rotation** `A' = Ry*A*(Rx*Rz)`. Modifying the OUTPUT matrix cannot feed back → follow + stick intrinsically safe. Earlier eye/target route (`+0xE0` / `+0xF0`) documented but superseded. **Ships `dxgi.dll`** (no dinput8 import); `.bind` = Steam DRM stub only, code in the clear.	✅
Metal Gear Rising METAL_GEAR_RISING_REVENGEANCE.exe	x86	6B+6A+6C	**Look-at rig.** Camera = `EAX` at `fld [eax+0x90]`; eye `+0x1B0`, lookat `+0x1C0` (Y-up). FOV encoding-aware (rad vs deg), base 66°. From IDK31's Camera Mod v1.1.2 CT, cross-validated against the exe. (Source header says "RememberMe.exe" — **stale comment only**; code uses `GetModuleHandleW(nullptr)`, README targets the right exe. §17.)	⚠

Ryu Ga Gotoku (old engine + Dragon Engine) & Persona

Game	Arch	Hook	Camera / offsets	St.
Yakuza: Like a Dragon	x64	6B VEH (HW-BP)	`pos[4]@0x00, focus[4]@0x10, rot[4]@0x20, pad[8], fov@0x50` — span `0x60`. Captured at `vmovups [rsi+0x80],xmm0` (RSI = struct). AOB `90 C5 F8 10 07 C5 F8 11 86 80 00 00 00` — write is at `match+5`; mask keeps `C5 F8 11 86 80 00 00 00` exact (86 = RSI, +0x80). Deltas precomputed by the worker, lock-free, so the handler does no I/O. **Gameplay detector before arming** (arming during load crashes). No modal popups.	✅
Yakuza Kiwami 2	x64	6A sync-cave	**Look-at rig.** position `+0x10`, focus `+0x20`, up `+0x30` (vec3+w each), fov `+0x58` (**radians**). Recovered by decoding etra0's kiwami2 freecam + disassembling the game. **Has an explicit up vector** — unlike other look-at rigs, so roll may be reachable here. Carries `WobbleAxis` + cutscene logic. 508 lines.	✅
Yakuza 6	x64	6B VEH	Dragon Engine family. Compact build (164→ small dllmain + proxy).	✅
Persona 5 Royal P5R.exe	x64	**pointer-chain** + 6B fallback	CORRECTED **Not a pure HW-BP mod.** Primary route: a global pointer loaded by a pattern-scanned instruction → `cam = *(*(global)+0x10)` → **4×4 view matrix @ `cam+0x08`**, projection @ `cam+0x60`. Each frame: read the game's matrix, post-apply the head pose, write back without compounding. A DR0/DR7 + VEH path remains as a **fallback** (`Dr7`: write, 8 bytes, L0). **The state-detector precedent** (§10): battle/cutscene by FOV window, sentinel scan, and motion — with per-axis strength + separate smoothing, hysteresis, and `BattleByMotion=0` by default because P5R's field camera pans as hard as battle. V2 = 439 lines (prefer it).	✅
Persona 4 Golden P4G.exe	x86	6B (discovery)	No anti-tamper → the HW-BP "find what writes" route works perfectly (fastest discovery of any title). Supports quat OR view-matrix (`applyMode`, `matTranspose`, `matTransAt=12`, `matWorldOrder`. **But struct writes were proven NOT to reach the rendered view. → Needs family 6E (D3D11 CB injection), which now exists in DXHR V2 as a working reference.** Rich F-key discovery tooling retained (F2 find-writer, F3/F4 matrix, F6 dump, F7/F8 quat, F9/F10 FOV).	"
Yakuza 0 Yakuza0.exe	x64	6A sync-cave Denuvo	**Old RGG engine. Camera lives in REGISTERS, not memory.** Hook RVA `0x18FD38`, steal **5** = `0F 29 64 24 40` (`movaps [rsp+0x40],xmm4`). At the hook: **xmm5**=focus, **xmm6**=eye, **xmm4**=up, **rax**=camera struct. FOV `+0xAC` **radians**. Cave spills xmm4/5/6, edits, reloads → the engine consumes our values downstream; no race. **Denuvo: 87% of the code section is INT3 at rest** → no static AOB is possible and the 5-byte pattern hits 93×. Located instead by **live-data probe** (§17.4). Proven with `RVA=0`.	✅
Yakuza Kiwami YakuzaKiwami.exe	x64	6A sync-cave	**Same engine as Yakuza 0 — and almost nothing transfers.** Hook RVA `0x30CC33`, steal **8** = `0F 29 4D E0 0F 29 45 D0` (`movaps [rbp-0x20],xmm1` ; `movaps [rbp-0x30],xmm0`) — **UNIQUE in .text**, a true version-agnostic anchor. At the hook: **xmm1**=focus, **xmm0**=eye, **[r9]**=up (**in memory**), **rbx**=camera object (vtable; next instrs are `mov rax,[rbx]` / `mov rcx,rbx` / `call [rax+0x208]`). **FOV is a PAIR:** `+0xAC` =radians, `+0xB0` =**tan(FOV/2)**, a cache the renderer reads. Write both (§15.12). `SetFov()` at `0x303C10`; NOP only its FOV store to hold cutscene FOV. **Not Denuvo** — the `.bind` section is only a SteamStub wrapper and the code is plain. Proven with `UseAOB=1`.	✅

RAGE (Rockstar)

Game	Arch	Hook	Camera / offsets	St.
RDR2	x64	6A sync-cave + 7D redirect	**The realized template — the reference implementation for new mods.** Hooks the camera-update function's **INPUT pov** (`rdi`) at IGCS `AOB CAMERA READ WRITE_INTERCEPT` (`44 0F 29 40 B8 44 0F 29 48 A8 44 0F 29 50 98 44 0F 29 60 88 44 0F 29 A8` @ func+0x28), steal **10**. Input pov: matrix `+0x00` (rows +0x00/+0x10/+0x20/+0x30), fov `+0x60`. **Redirect-the-copy** (§7D): 8 heap buffers, copy 0x100 bytes, rotate the copy, `mov rdi,rax` — the game's stack is never touched (this fixed the long-session freeze). 16-slot per-pointer cache, `SmoothCD` on all 6 channels, **both wobble modes**, radians-aware FOV, Gram-Schmidt re-orthonormalization. Rotating the input BEFORE the function processes it moves the real view while the character keeps moving normally.	✅
GTA V Enhanced GTA5_Enhanced.exe	x64	**6F native** (Mode 5) **+ 5A** (Mode 10)	**The hardest target; the most advanced mod (1667 lines).** Arxan + DX12 + camera-relative rendering + logic≠render camera. **Mode 5**: scripted cam via natives — full 6DOF in gameplay, but the cutscene director owns the camera. **Mode 10 (the breakthrough)**: trampoline-hook the **generic "copy pov into camera"** function `0x1ae150` (AOB `56 57 48 83 EC 28 48 89 D7 48 89 CE 48 83 C2 10 F3 0F`, wildcard-free), and edit the **source pov** gated to `dest == g_hObj − 0x10` so only the render camera is touched → tracks in **gameplay AND cutscenes from one hook**. Struct: rot `+0x10`, pos `+0x40`, fov `+0x70`. Render camera located every session by **behavioural finder** (orthonormal + moves + next to a projection, matched to `GET_FINAL_RENDERED_CAM_COORD`) + watchdog re-lock + camera-cluster cache. Spring smoothing. DX12 Present hook exists but is **opt-in, default OFF, untested**. **Two packages**: `...0` = 643,578 B / 125,036-line src (older); `...` = 644,803 B / 128,574-line src (newer — **prefer**).	✅
GTA IV	x86	RTTI vtable	`TheCamera → +0x04 m_pFinalCam`; `CCam: +0x10 m_mMatrix` (rows right/up/at/pos, `CVector_pad` = 4 floats each), `+0x60 m_fFOV`. **`CCamFinal` vtable resolved by RTTI** → version-agnostic. iv-sdk offsets, native, no ScriptHook. **The key doc note**: "the logical `m_mMatrix` is 0x10 but the game recomputes it; **0x510 is the composited matrix actually drawn**" — this is the insight that unlocked GTA V.	✅

UE3 / REDengine / CryEngine / MT Framework / other

Game	Arch	Hook	Camera / offsets	St.
Batman: Arkham Origins	x86	6D writer + auto-POV	`FVector loc +0x0/4/8`, `FRotator +0xC/0x10/0x14`. **FOV-hook identifies the camera**; **auto-discovers the POV offset** at runtime (`dest − cam < 0x2000`). Carries `WobbleAxis`.	✅
Batman: Arkham City	x86	6C inline (same writer)	POV ~`+0x3AC` in City; Origins auto-discovers it.	✅
Batman: Arkham Asylum	x86	6B VEH	UE POV block.	✅
Batman: Arkham Knight BatmanAK.exe	x64	6A inline cave	`cameraStruct +0x574` (rot/pos block). Applied on the game's camera thread only → no races/flicker. **cameraunlock-core smoothing model**: framerate-independent exponential baseline (`kBaselineSmoothing = 0.15`; <0.001 snaps, else speed lerps 50→0.1). **Recenter captures a smoothed baseline**, not a raw snapshot → HOME is jitter-free. Split into `dllmain/camera_hook/config`. (V2 package is **binary-only, no source** — §17.)	✅
The Witcher 3 witcher3.exe	x64	FOV-identify hook	**Euler DEGREES**: roll `+0x80`, pitch `+0x84`, yaw `+0x88`; pos `+0x70` (left alone). **+0x70 is NOT a quaternion** — logged as `323.6/280.5/17.5/1.0`; the classic wrong turn. FOV-write capture (`rcx → g_realCam`) is the reliable way to pick the *gameplay* camera, avoiding cutscene/menu cameras. Calibration slots for +0x70/74/78 if a build shifts.	✅
The Witcher 2 witcher2.exe	x86	6C inline (matrix)	Hooks the camera VIEW-matrix **READ** that Heebo's 2022 freecam uses: AOB `F3 0F 10 2E F3 0F 10 5E 24 @ +0x12473`. The engine reads its 4×4 from `[esi]` here, **only for the main camera when `ecx == 0`**. Rotate in-frame right before the engine consumes it → culling/render/shadows all follow, no timing race. Ships `inlinehook_x86.h` (the reusable x86 primitive) and `WobbleAxis`; **wobble default ON**.	✅
Crysis 3 Remastered	x64	runtime scan	Native CryEngine. Runtime-scans for the view `CCamera`'s **`Matrix34`** (version-agnostic); additive nudge + FOV. Carries `SmoothCD`. 499 lines, `v0.1 camera-finder`.	⚠
Dead Space Remake (2023)	x64	6A sync-cave	Built on the TR2013 matrix mod + SOTTR's 64-bit cave. Pointer-chain: AOB `A1 ?? ?? ?? ?? 85 C0 75 ?? 53 → camPtrGlobal = *(match+1) → cam = *camPtrGlobal →` 4×4 VIEW `@ +0x08`. Steal 8 (`49 8B C6 48 8B 74 24 58`). **Origin of the `g_applyGate` rate-limiter** and the **6-axis named wobble sweep**. FOV Scale/Degrees.	✅
Resident Evil 6 BH6.exe	x86	6C inline	MT Framework. AOB `F3 0F 10 81 90 00 00 00 F3 0F 11` (`movss xmm0,[ecx+0x90]`) → `ecx` = camObj. **4×4 world→view** @ `camObj+0x90`, row-major, built as `trans(-eye)*R`; projection just after (`proj[0][0] ≈ camObj+0xD0`). Head rotation = **post-multiply** (`V' = V*H`); lean = **subtract** camera-space lean. Carries `WobbleAxis`. From Heebo's CT + the exe.	✅
Resident Evil 5	x86?	—	**Binary + INI only** — no source, no README (§17). `.asi` 395,551 B; `version.dll` proxy.	⚠
MGS5: TPP mgsvtpp.exe	x64	6C inline (in-path)	From Otis IGCS. The game writes the camera via a tiny function: `movaps xmm0,[rdx]; movaps [rcx+0xF0],xmm0; movaps xmm1,[rdx+0x10]; movaps [rcx+0x100],xmm1; ret`. `rcx` = struct: **quat `+0xF0`, pos `+0x100`, fov `+0xC`**. X=right, Y=up, Z=out-of-screen. Inline-hook that write; multiply the quat by the head quat + add camera-local lean right after the game writes → source-of-truth camera, no smear. **SteamStub-packed** → scan the **decrypted** code at runtime after a delay; pattern is **register-agnostic** for version independence.	✅
Crimson Desert CrimsonDesert.exe	x64	6A (CDPatchCamera)	**The hard-case reference (§13).** BlackSpace (Pearl Abyss), packed/protected. Rotation **WORKS** via quat `+0xD8` at `vmovups [r12+0xC8],ymm0` (EXE `0x140e2dafe`) — same site Otis labels `AOB_CAMWRITE INTERCEPT`. **Positional lean unresolved**: the spring-arm recomputes the render position; `camBase+0xC8` and `finalBase+0x10` both proven dead by the constant-nudge probe. Render transform = **vtable-returned**: quat `+0x00`, pos `+0x10/14/18`, via `camera@[rsi+0x328] → call [vtable+0x18]`; AOB `48 8B 8E 28 03 00 00 48 8B 01 FF 50 18`. REV24 shipped awaiting test. **Fallback: ship rotation-only** — it's the bulk of the effect.	▪ rot-only
CamHook Toolkit	—	6A universal	One DLL that injects (`CamHookInjector.exe`) or loads as a proxy. Finds a game's camera and applies additive 6DOF **without per-game code**. Defaults `offPos=0x60`, `offQuat=0x70` (override in `CamHook.ini`). Shared math: `qfromEuler(yaw,pitch,roll,mode)`, `qrotvec(quat,v)`, `IsUnitQuat()`. Lean map: `quatMode==1 → leanL=(dx,dz,dy)` (Z-up) else `(dx,dy,dz)`. Discovery hotkeys: F5 vtable scan, F6 dump struct, F7/F8 quat+delta, F9/F10 FOV+delta, INSERT arm/cycle.	✅

24. Per-engine playbook

Engine	Start from	What to expect
AnvilNext 2.0	AC Origins (Denuvo) / AC Unity (chain)	Quaternion + mirror copies. Denuvo bans DRs → 5A. **Write both the primary and the mirror.** Hook the LAST write of the frame.
Foundation	SOTTR	coords/quat/fov at `+0x80/+0xA0/+0xB0`, steal 34. Many cameras through one site → add the address-select cave.
Frostbite 3	DA: Inquisition	Duplicate rot+pos copies. **Write both.**
Eclipse	DA: Origins (x86 VEH) / DA2 (x86 inline)	eye/proj/VIEW/WORLD/fov at `+0x10/+0x40/+0x80/+0xC0/+0x140`. FOV field may read zero → `NoFov=1`.
Northlight	Alan Wake Remastered / Control	**Tight 3×3 stride 0x0C**, pos `+0x24`. Pooled/transient cameras → hook the writer *instruction*, never an address. Pitch may be re-derived downstream → hook the final view setter. Cutscene by hook-staleness.
Dawn / Glacier	DXMD → Guardians	Row-major 3×3 + position, Z-up / Y-into-screen. `YawMultiplier=-1`, `InvertY=1`, `WorldUnitsPerMetre=10`. Packing differs (tight vs vec4-padded) but the logical content is identical.
Dragon Engine	Yakuza LAD (VEH) / Kiwami 2 (look-at)	Compact structs. Arming a HW-BP during load crashes → gameplay detector first.
PlatinumGames / Level-5	NieR Automata / Ni no Kuni	**Look-at rigs.** Reset cameras → absolute-on-live. No roll DOF. Watch for pointer fields and live-zoom fields. Prefer the **output view matrix** over eye/target where one exists.
UE3	Batman Arkham Origins	POV memcpy; `FRotator` ints (65536 = 360°). FOV-read hook IDs the camera; auto-discover the POV offset.
REDengine	Witcher 3 (Euler) / Witcher 2 (matrix)	W3 stores **degrees**. FOV write identifies the gameplay camera.
RAGE	RDR2 → GTA V Enhanced	Input-pov layout: rot `+0x00`, pos `+0x30`, fov `+0x60`. **The drawn matrix is a composited one, distinct from the source frame.** Edit the pov at the build site, in-thread.
MT Framework	RE6	4×4 world→view at `camObj+0x90`, projection right after.
Spring-arm 3rd-person	Crimson Desert (§13)	Additive writes to upstream copies are invisible. Target the vtable-returned transform, or saturate the read sites. **Never disable the spring-arm.** Rotation-only is a legitimate ship.

25.0 The first 30 minutes on a new game NEW

Everything in this document, compressed into the order you actually do it. Each step is cheap and each one can eliminate days of the next.

Min	Do	Because
0–1	**Read the exe's filename.**	`*-Win64-Shipping.exe` / `*-WinGDK-Shipping.exe` → **22% of games; camera already solved** (§15.1). Skip to minute 20.
1–2	**List the exe's imports.**	Picks the proxy name. `dinput8` (29/49 shipped) → `version` (13) → `dxgi` / `winmm` / `wininet` / `dbghelp`. **Wrong proxy = the mod never loads**, and it looks exactly like a broken hook.
2–3	**Check `.text` entropy, section names, and `.pdata`.**	Packed/protected → the AOB must be scanned **after** decryption, at runtime, on a delay. Also decides Denuvo/Arxan → **no debug registers** → family 6A. **Read `.pdata` even on a protected exe — it usually survives** (see below).
3–5	**Search the corpus** (§20).	Constant header → offsets + **steal length** in plain text. Table → AOB + full original bytes. GPU-only profile → **stop; go to 6E**, there is no CPU camera.
5–10	**Decide the three axes:** hook family (§6) × camera math (§7) × drive model (§8).	Getting the **drive model** wrong is the expensive one — it produces a mod that tracks but feels wrong, and it looks like a tuning problem for days (§8).
10–20	**Capture and dump.** Hook the address site, capture **RCX first, then RDX** (52% between them), and dump **0x80 bytes** as hex+float once per second.	**94% of camera fields are within 0x74** (§15.9). One 0x80 dump next to a known head pose answers the layout.
20–25	**Identify the struct** from the dump: find position (3 plausible world floats that change as you walk), then test **+0xC / +0x10 / −0x10** for orientation.	Three cases cover **73%** (§16). Then read the validators: roll ≈ 0, w ≈ 1.0 or 0.0, aspect 1.7778 (§17.2).
25–30	**Wobble.** `WobbleMode=1`, all six axes, no tracker.	If the whole view pans by itself, **the hard part is done**. If not, you learned that in 30 minutes instead of three days (§10, §13).

THE SEVEN QUESTIONS, IN COST ORDER

```
Q1  Is the exe named *-Shipping.exe?          -> 10 seconds. Solves 22% of games.
Q2  What does the exe import?                 -> 1 minute.  Decides the proxy.
Q3  Is it packed / anti-tampered?             -> 1 minute.  Decides the hook family.
    (and does .pdata survive? usually YES, even under Denuvo)
Q4  Is it in the corpus?                      -> 2 minutes. May solve it outright.
Q5  Does a CPU camera exist at all?           -> 2 minutes. 131/209 GPU-profiled
                                                 titles have NO CPU camera.
Q6  Is the camera reset or persistent?        -> 10 min.    Decides the drive model.
Q7  Does writing the struct reach the screen? -> the wobble test. THE only proof.

Q1-Q5 cost under ten minutes combined and can each save days.
Nobody skips them twice.
```

25.0.1 The eight ways a bring-up actually fails

Ranked by how much time they have cost across the library. All are cheap to rule out early and expensive to discover late. **6–8 are new, and 6 and 8 are the ones that masquerade as something else.**

#	Failure	Tell & fix
1	**Editing an upstream copy**	**Perfect diagnostics, nothing on screen.** The engine recomputes downstream. Find the copy the renderer reads (§13, §8D). **The most expensive single failure in the project's history** — it is what the flagship's `0x510` /composited-matrix insight and the hardest title's vtable transform are both about.
2	**Wrong drive model**	Tracks, but fights you / drifts / feels rubbery. Reset camera driven incrementally. → absolute-on-live (§8).
3	**Wrong steal length**	**Instant crash on install.** The `E9` landed mid-instruction. Take the length from `CONTINUE − START` if a constant header exists (§15.6); otherwise decode instruction boundaries properly — never guess.
4	**Right camera, wrong one of several**	Works in gameplay, breaks in cutscenes/menus (or vice versa). → the gate + a 16-slot per-pointer table (§8E), or hook the **last** write of the frame.
6	**A signature lifted from someone else's shellcode is off by 8** NEW	**Presents exactly like "wrong game version" — which is what makes it expensive.** Before trusting a re-emitted instruction from a reference tool, ask **how that tool entered**. A shellcode ending in `ret` was entered by `CALL` (`E8 rel32` = 5 bytes, a perfect fit over a 5-byte steal), which **pushes 8 bytes of return address**. So **every rsp-relative displacement in its replayed instruction is +8 versus the game's real code**. One real case: the shellcode showed `movaps [rsp+0x48],xmm4`; the game actually executes `movaps [rsp+0x40],xmm4`. The author's own comment said "adjusted offset of stack pointer + 8" — the answer was in the source the whole time. **The check is free:** is the displacement rsp-relative? Then subtract 8 if they entered by CALL. **RBP-relative displacements are unaffected** (rbp is untouched by the push) — a sibling title on the same engine had an rbp-relative steal and its bytes were literal. Entering by `JMP` and restoring rsp exactly, then replaying the original bytes *verbatim*, sidesteps the whole class.
7	**Inheriting a sibling title's constant unverified** NEW	Same engine buys you the **shape** — rig type, roughly where FOV lives — and **nothing else**. Two titles on one engine differed in *every* register role (focus/eye/up/object all different), steal length (5 vs 8), and base register (rsp vs rbp). Worse, the reference tool for the second title had **copy-pasted one address from the first and never updated it**. Every other constant was correctly re-derived — including the neighbouring patch's bytes — so the error was invisible by inspection. At that address the second game has unrelated code, and the tool **NOPs it on toggle and then "restores" bytes that were never there**, corrupting the game. Verified by disassembling the target: 3 of its 4 constants matched the exe byte-for-byte, which **proves the build is right and the 4th constant is simply wrong**. **Rule: re-verify every inherited constant against the target binary, and locate by AOB rather than by an address you were given.**
8	**Your verification never actually ran** NEW	**The most dangerous failure here, because the output is encouraging.** Two real cases in one session: (a) a config key that was read from the INI and **never referenced** — setting it changed nothing, and the mod's success "confirmed" a path that never executed; (b) a **stale binary** — the test ran against the previous build. Mitigations, all cheap: **put a build-identifying string in the startup log** so one line answers "am I running what I think I am"; **assert every INI key is referenced in the source** (a 5-line script over the INI + source catches dead keys instantly); and make diagnostics **compare** rather than assume — one diagnostic here checked only for `0xCC` and otherwise always printed "build differs", contradicting the verifier two lines later. **A confidently wrong log line is worse than no log line.**
5	**Struct writes never reach the GPU**	Struct changes, screen doesn't, and the value survives a frame. → family 6E. **Check for a GPU-only profile first — this is 63% of GPU-profiled titles.**

25. Step-by-step: making a NEW mod

1. **Look the game up in the corpus FIRST** (§20). Engine sibling → you already have the AOBs (§15.1). Constant header → offsets in plain text (§15.6). GPU-only → go straight to 6E (§18). Also check the exe's **imports** (proxy name) and `.text` **entropy** (packed?).
2. **Choose the skeleton** — copy the closest existing mod of the same family + math:
   - x64 quaternion sync-cave → **SOTTR**
   - x64 matrix → **Dead Space** / **Guardians**
   - x64 input-pov / general modern template → **RDR2** ← *the best starting point*
   - x86 matrix inline → **Witcher 2** (`inlinehook_x86.h`)
   - UE3 writer-hook → **Batman Origins**
   - look-at rig → **NieR Automata** / **Ni no Kuni**
   - struct writes don't reach the screen → **Deus Ex HR V2** (5E)

   Reuse `opentrack_receiver.{h,cpp}` + `logger.h` unchanged (take the richest variant, §3).
3. **Fill the [Hook] block:** AOB (with `??` wildcards), steal length, capture register, struct offsets.
4. **Ship wobble-first:** `TestWobble=1` , `WobbleMode=1` (full 6-axis sweep). Build, run, get in-world.
   - Whole view pans by itself → AOB is correct. Step each axis.
   - Nothing pans / wrong object → §13, or add the address-select capture cave.
5. **Flip to 6DOF:** `TestWobble=0` . Add rotation (axis map + inverts), then lean, then FOV. Tune sensitivity/clamps/lean scale.
6. **Pick the drive model** (§7) — this is where "tracks but feels wrong" is decided. Reset or persistent?
7. **Add the polish:** spring smoothing default, recenter-from-first-packet, dummy-camera rejection, per-pointer slot table, single-instance mutex, last-good cache, warm-up gate, discontinuity guard, cutscene detector if the FOV field is valid.
8. **Build & package:** compile with the §4 line (win32 threads); verify size; zip `asi + proxy dll + ini + license + README + src/` .

Milestone checklist per mod

	Milestone
☐	AOB scans and hooks (log shows the match address); install **verifies bytes and aborts safely** on mismatch
☐	Wobble pans the whole view on desktop — **all six axes** (`WobbleMode=1`)
☐	Rotation tracks head correctly (axis map + inverts dialed; split-axis if it skews)
☐	Lean moves the viewpoint the right way (per axis); no distance-system fight
☐	FOV option works (scale / absolute / cutscene keeps its own); encoding correct (deg/rad/tan-half)
☐	Smoothing removes jitter (spring default, `SmoothHz=8`); `dt` clamped; state seeded
☐	Recenter (HOME) and toggle (END) work; no start-tilt (recenter from first real packet)
☐	Survives map/cutscene/save transitions (no stall, no flip-flop, no crash)
☐	Single-instance mutex; receiver starts only after the hook installs
☐	Rebuilds to the same size; packaged with source + INI + README + license

26. Library audit — gaps & drift

Findings from the word-by-word audit of the 51 shipped packages. None are functional bugs in a working mod; all are packaging/consistency debt worth clearing.

26.1 Feature coverage (measured)

Feature	Coverage	Mods
Wobble test (any form)	14 / 51	—
Per-axis `WobbleAxis`	**8**	Batman Origins, Witcher 2, RDR2, GTA V (×2), RE6, TR2013, Yakuza Kiwami 2
Full 6-axis sweep	**2**	RDR2 (`WobbleMode=1`), Dead Space
Spring smoothing	**5**	RDR2, TR2013, Crysis 3, Quantum Break, GTA V (as `spr()`)
Cutscene logic	8	Control (×2), DXMD v2, GTA V (×2), NieR Automata, P5R V2, Yakuza Kiwami 2

Read: the template-era mods (RDR2, GTA V, Witcher 2, Kiwami 2, TR2013) carry the modern kit. Older ones predate it. **The standing goal — fold spring + 6-axis wobble + generic cutscene-own-FOV + standardized static FOV into every mod — remains partial.** RDR2 is the donor for all four.

26.2 Packaging gaps

Package	Issue
RE5-HeadTracking	**Binary + INI only.** No source, no README, no license. Cannot be rebuilt or maintained.
BatmanArkhamKnight V2	**Binary-only** (asi + dll + ini + license). No `src/` . The V1 package has the source.
NieRReplicant (two packages)	**Split:** `-v0` has the `.asi` but the other has the source. Neither is complete alone.
DXMD v2	A stray duplicate `.asi` inside `src/` as well as the package root.
Duplicate/variant packages	AC Unity (+Smooth), Alan Wake 2 (+V2), Batman AK (+V2), Control (+steam), DXHR (+V2), DXMD (+v2), DA2 (+V2), DAO (+V2), GTA V (+0), NieR Replicant (+v0), P5R (+V2) — **11 pairs**. The newer/better one isn't always obvious from the name (e.g. GTA V's better build is the one *without* the `0`).

26.3 Framework drift

- `opentrack_receiver.h` : **8 distinct variants**. `logger.h` : **8 distinct variants**. Nominally "shared, byte-identical"; not.
- The richest receiver (`LatestPredicted` + `Packets()`) is in DA2, DAO, Witcher 2, GTA V only.

26.4 Stale headers (cosmetic, library-wide)

Mods are built by copying the nearest sibling as a skeleton; the top comment block — and occasionally an in-code default string the INI overrides — sometimes isn't updated.

Mod	Stale reference	Functional?
Metal Gear Rising	Header line 5: "Target: RememberMe.exe (x86, UE3, D3D9)"	**No** — code uses `GetModuleHandleW(nullptr)` ; README correct. RESOLVED **Origin identified:** the corpus contains 15 scan tables targeting `RememberMe.exe` . The mod was skeletoned from a table-derived x86/UE3 build; the comment is a fossil of that lineage, not an error.
Dragon Age 2	DAO header + `moduleName` default `"DAOrigins.exe"`	**No** — INI overrides
NieR Replicant	Alan-Wake-2 header	**No**
Ni no Kuni	NieR-Automata header	**No**

In every case the runtime name, offsets, AOBs, RVAs and hook method are correct for the actual game. Worth a one-pass comment cleanup; nothing functional.

26.5 Documentation contradictions

> **GTA V `TECHNICAL-REFERENCE.md` §3b is stale.** It states the RDR2/legacy camera code-site AOBs return **0 hits** on Enhanced and concludes "this rules out the RDR2-style code-cave inline hook... no public tool has a signature." `METHODS.md` §3.10–4 then documents the **working** `0x1ae150` trampoline, found later via the decrypted runtime dump + the `PAGE_READONLY` find-what-writes. **`METHODS.md` is authoritative.** The reference doc predates the breakthrough and should be reconciled.

26.6 Docs living outside the MOD DOCS bundle

Six per-game findings docs are inside mod folders and were never in the docs bundle or Rev-1 — they contain some of the project's best material and are folded into this document:

- **FINDINGS-NieRAutomata.md** (18 KB — the largest; absolute-on-live, look-at roll proof, crash guards, IGCS RE)
- **FINDINGS-NinoKuni.md** (10.7 KB — the five-pass drive-model saga, split-axis rotation)
- **FINDINGS-AlanWakeRemastered.md** (9.9 KB — proxy imports, tan-half FOV, stride 0x0C, AllocNear fix)
- **FINDINGS-GuardiansOfTheGalaxy.md** (6.6 KB — IGCS extraction method, Dawn-engine defaults)
- **FINDINGS-ACOrigins.md** (2.8 KB, in-package — the extended version; WRITE2 reasoning)
- **FINDINGS-NieRReplicant.md**, **CHOPPINESS-FIX.md**, **VERSION-AGNOSTIC-FIX.md**

26.7 The shared core library that was never extracted MEASURED

Scanning function definitions across all **47** `dllmain.cpp` shows a **de-facto standard library** that every mod re-implements by copy-paste. It has never been extracted into a shared header, and it has drifted:

Primitive	Mods	Role & drift
`LoadCfg`	**25**	INI reader. **Also spelled `loadCfg` (8) and `LoadConfig` (7)** — 40/47 mods, three names.
`Readable` / `Writable`	**22 / 19**	**The most valuable primitive in the library.** `VirtualQuery`-based safety probes — the MinGW answer to having no SEH (§5). Byte-identical everywhere; belongs in a header.
`Clamp`	19	Trivial, duplicated 19 times.
`AobScan`	**18**	**Four different signatures**: `(pat,n)` · `(pat,mask,size_t patlen)` · `(pat,mask,size_t n)` · `(pat,mask,int plen)`. Same function, four APIs.
`InstallHook` / `RemoveHook`	15 / 12	Cave install/uninstall.
`Down`	14	Hotkey edge-detect.
`ModuleRange`	8	Module base+size for the scan.
`Smooth` / `SmF` / `EffSm` / `Lerp` / `advancePose`	7 ea.	**The smoothing filter, under five different names.** This is why spring smoothing is only in 5/51 mods — there was no one place to add it.
`FrameDt`	7	Frame delta with the [0.5 ms, 100 ms] clamp.
`mul3` / `qrotvec`	8 / 6	Vector/quaternion math.
`LooksLikeMatrix`	6	Validator (§17.2).
`rdRow` / `wrRow`	6	Matrix row access with configurable stride (0x0C vs 0x10).

The recommendation, and the reason it matters more than tidiness. Extract `ht_core.h` alongside `opentrack_receiver.h` and `logger.h`, containing exactly these primitives under one canonical name each. **The measured cost of not having it is visible in the coverage numbers (§26.1):** spring smoothing reached only 5/51 mods and the 6-axis wobble only 2/51 — **not because they were hard, but because there was no shared place to put them.** Every improvement to this project currently costs 51 edits, so it gets made once and never propagates. A core header turns "fold the modern kit into every mod" from a 51-mod migration into a header bump plus a per-mod recompile.

27. Quick reference — constants

Item	Value
OpenTrack UDP port	`4242`
OpenTrack packet	6 LE doubles (48 B): `x,y,z` (cm → m ×0.01), `yaw,pitch,roll` (deg)
Toggle / recenter hotkeys	`END / HOME`
Proxy loader names (measured, 49 shipped)	`dinput8.dll` **29** · `version.dll` **13** · `dxgi.dll` **3** · `winmm.dll` **2** · `wininet.dll` **1** · `dbghelp.dll` **1** — **match the exe's imports**. dinput8+version cover 86%; the rest exist because the exe imported neither.
x86 build	`i686-w64-mingw32-g++-win32 -O2 -std=c++17 -shared -static -static-libgcc -static-libstdc++ -lws2_32 -lwinmm -luser32`
x64 build	`x86_64-w64-mingw32-g++-win32 … -lws2_32 -lpsapi`
Threads model	**win32** (posix changes binary size: DAO 384,690 → 536,252)
rel32 cave reach	±2 GB (`AllocNear` must walk `MEM_FREE` ; 14-byte `FF 25` fallback)
Win64 call prep	align rsp to 16, +0x20 shadow space, save xmm0–5
Spring default	`SmoothHz ≈ 8` (critically damped), all 6 channels; clamp `dt` to [0.5 ms, 100 ms]
Slot table size	16 (per-pointer clean base)
Wobble defaults	`WobbleDeg=15` , `WobblePos=0.3` , `WobbleHz=0.5` , dwell 2.5 s, 6 axes
Camera layouts	
DAO / DA2	eye `+0x10` , proj `+0x40` , VIEW `+0x80` , WORLD `+0xC0` , near/far `+0x130` , fov `+0x140`
SOTTR / ROTTR	coords `+0x80` , quat `+0xA0` , fov `+0xB0` , steal 34
AC Origins	pos `+0x60` (mirror `+0x2E0`), quat `+0x70` (mirror `+0x2F0`), fov `+0x264`
AC Unity / Syndicate	transform = `camObj+0x50` : pos `+0x00` , quat `+0x10` , fov `+0x20`
Yakuza LAD	pos `@0x00` , focus `@0x10` , rot `@0x20` , fov `@0x50` , span `0x60`
Yakuza Kiwami 2	pos `+0x10` , focus `+0x20` , up `+0x30` , fov `+0x58` (rad)
Witcher 3	roll `+0x80` , pitch `+0x84` , yaw `+0x88` (**Euler degrees**), pos `+0x70`
Guardians	rows `+0x20/+0x30/+0x40` , pos `+0x50` , extra `+0x60` (`rbx` = camera, steal 38)
NieR Automata	eye `+0xF0` , target `+0x100` , dist `+0x134` , fov `+0x138` (rad), fovTgt `+0x188` ; **never** `+0x60` or `+0x130`
Ni no Kuni	view matrix `@+0x000` , coords `@+0x030` (hook `0x5B5CF8` , steal 28)
MGS5	quat `+0xF0` , pos `+0x100` , fov `+0xC`
RE6	4×4 world→view `@camObj+0x90` , proj ≈ `+0xD0`
Alan Wake Rem.	rows `+0x00/+0x0C/+0x18` (**stride 0x0C**), eye `+0x24`
RAGE input pov	rot `+0x00` , pos `+0x30` , fov `+0x60`
GTA V render cam	rot `+0x10` , pos `+0x40` , fov `+0x70` ; hook `0x1ae150` ; gate `dest == g_hObj - 0x10`
GTA IV	`TheCamera+0x04` = `m_pFinalCam` ; `CCam+0x10` = matrix, `+0x60` = FOV; **`0x510` = composited drawn matrix**
Crimson render transform	quat `@+0x00` , pos `@+0x10/0x14/0x18` (via camera `vtable+0x18`)
Batman AK	`cameraStruct+0x574`
Quantum Break	**12 doubles** @ `camStruct+0x70`
Deus Ex HR	`*(base+0x187CEC0) → +0x30 → view @ +0x80` ; CB hook Map 14 / Unmap 15 / UpdateSubresource 48

Corpus constants (Part III)

Item	Value
CALL-entered shellcode	**rsp-relative displacements in its replayed instruction are +8** vs the game's real code (the CALL pushed a return address). **rbp-relative are unaffected.** A shellcode ending in `ret` was entered by CALL.
FOV may be a PAIR	`+0xAC` raw radians **and** `+0xB0` = **tan(FOV/2)** cached. Write BOTH. A QWORD move on a float FOV field is the tell. `+0xB0 ≈ tan(+0xAC/2)` is the strongest validator available.
`.pdata` under anti-tamper	**Survives.** One Denuvo exe was 87% INT3 in its code section yet carried **65,628 RUNTIME_FUNCTION records, 99.9% sorted and plausible** — a free map of every function boundary. Always read it.
Protected-exe reality	**87% INT3 at rest** → no static AOB, no static verification. A 5-byte spill signature hit **93×**. Identify by **live data** (§17.4), not by code.
Same engine, different title	Buys the **shape only**. Observed to differ: every register role, steal length (5 vs 8), base register (rsp vs rbp), and whether up is in a register or memory. **Re-verify every inherited constant.**
Camera struct size	**70% of fields within +0x30 · 94% within +0x74.** Dump **0x80 bytes** and you have the whole camera.
Camera pointer register	**RCX 26.2% · RDX 26.1%** (=52%) · RAX 7.9% · RBX 6.4%. **Try RCX, then RDX.** Win64 fastcall: RCX = `this` /dest, RDX = arg2/src.
Top camera offsets	`+0x08` 14.6% · `+0x10` 12.3% · `+0x18` 11.7% · `+0x0C` 10.6% · `+0x14` 7.9% · `+0x20` 7.1% · `+0x30` 6.3% · `+0x04` 4.3% (**top 8 = 75%**)
Engine from filename	`*-Win64/WinGDK-Shipping.exe` = **58 of 261 modules (22%)** → AOBs already known
FOV encoding by value	**30–160 = degrees** (dominant; native default often 75, usually horizontal) · 0.5–2.8 = radians · 0.3–1.7 changing with zoom = tan-half · reads 0.0 = not the FOV field
Camera hook = SSE store on `[reg+disp]`	**42%** of all camera sites (61% incl. SSE loads). LEA `[base+disp]` = **10.6%** = the address capture. **vtable = 0.7%**. x87 = 3.9% (32-bit only).
Steal length	**No standard.** Observed 6, 7, 8, 12, 16, 17, 21, 60, 87, 104 B. Take it from `CONTINUE − START` when a constant header exists.
Orientation convention split	quaternion **44%** · look-at **34%** · matrix **22%** (41 deduped games)
Engine-sibling POV write AOB	`F2 0F 11 01 48 8B DA 8B 42 08 89` (41 tables; `rcx`=dest, `rdx`=src; steal 35)
Engine-sibling address AOB	`48 8D 91 ?? 1? 00 00 48 8B CB E8 ?? ?? ?? ?? 48 8B C3 48 83 C4 20 5B C3` (32 tables)
Engine-sibling POV struct	loc `+0x00/04/08` · rot Euler **degrees** `+0x0C/10/14` · fov `+0x18`
Managed-runtime writes	pos `[obj]` · quat `[obj+0x10]` ; gate on captured obj
The three struct layouts	Δ(rot−pos) = **+0xC** (10) · **+0x10** (8) · **−0x10** (6) — **24 of 33 deduped games (73%)**. (+0x30 rot-first variant: 4.)
Validator fingerprints	homogeneous w `[0.9999,1.0001]` · vec4 pad `[-0.001,0.001]` · roll≈0 `[-0.001,0.001]` (56 games) · pitch `[-π/2,π/2]` · yaw `[0,2π]` wrap · aspect `[1.77777,1.77779]` =16:9, `[1.33332,1.33334]` =4:3
ScanType block classes	**1xxx**=Rotation · **2xxx**=Position · **3xxx**=FOV
CB matrix	**64 bytes** (4×4) in 165/166 · slot **0** dominant · byte offset **0** dominant · transpose needed ~27% · `TransformUpdateMapType=4` (WRITE_DISCARD)
Shader matrix name	`proj` / `Proj` / `PROJ` (241/395), then `ViewProj` / `WorldViewProj` / `mvp` variants
Corpus priors	sensitivity median **0.86** (mode 1.0) · positional enabled **315/370** · invert False **365/370** · crouch **−0.1 m** / jump **+0.05 m**
GPU-only titles	**131 of 209** have no CPU camera → family 6E is the only route

End of the project bible, Revision 4. Rebuilt July 17, 2026 from a word-by-word audit of 51 shipped packages, the MOD DOCS bundle, six in-package findings docs, the flagship method logs, and the camera-data corpus (345 unique scan tables / 882 unique AOBs / 539 classified camera sites, 49 constant-header games, 820 GPU+scan profiles, 5 aggregate CSVs), and a key-by-key audit of all 51 shipped INIs. Captures the framework, six hooking families, four camera-math conventions, the drive-model taxonomy, version-agnostic mechanisms, the full-6DOF wobble self-test, FOV + four cutscene detectors, smoothing, a 22-entry failure-mode catalogue, the per-game atlas, the per-engine playbook, the library audit, INI conventions and a step-by-step recipe for new mods. To resume in a new chat: drop this PDF in, say "continue from this reference", name the game, and start at §20 (corpus lookup) → §25 (the recipe).